Contents

1 Choose Your Destiny Manual

Figure 1: logo

The program is a compiler and interpreter to run "Choose Your Own Adventure" type stories or choice-based adventures and game books, for the Spectrum 48, 128, +2 and +3.

It consists of a virtual machine that interprets commands found in the text to perform the different interactive actions and a compiler that is responsible for translating the adventure from a BASIC-like language, with which the adventure script is written, to a file interpretable by the engine.

In addition, it can also display compressed images stored on the same disk, as well as sound effects based on Shiru's BeepFX and PT3-type melodies created with Vortex Tracker.

* OPTION GOTO ID
* OPTION GOSUB ID
* OPTION VALUE(varexpression) GOTO ID
* OPTION VALUE(varexpression) GOSUB ID
* CHOOSE
* CHOOSE IF WAIT expression THEN GOTO ID
* CHOOSE IF CHANGED THEN GOSUB ID
* OPTIONSEL()
* NUMOPTIONS()
* OPTIONVAL()
* CLEAROPTIONS
* MENUCONFIG varexpression, varexpression, varexpression, varexpression
* MENUCONFIG varexpression, varexpression, varexpression
* MENUCONFIG varexpression, varexpression
* CHAR varexpression
* REPCHAR expression, expression
* TAB expression
* PRINT varexpression
* PAGEPAUSE expression
* INK varexpression
* PAPER varexpression
* BORDER varexpression
* BRIGHT varexpression
* FLASH varexpression
* NEWLINE expression
* NEWLINE
* BACKSPACE expression
* BACKSPACE
* SFX varexpression
* PICTURE varexpression
* DISPLAY varexpression
* BLIT varexpression, varexpression, varexpression, varexpression AT varexpression, varexpression
* WAIT expression
* PAUSE expression
* TYPERATE expression
* MARGINS expression, expression, expression, expression
* FADEOUT expression, expression, expression, expression
* AT varexpression, varexpression
* FILLATTR varexpression, varexpression, varexpression, varexpression, varexpression
* PUTATTR varexpression, varexpression AT varexpression, varexpression
* PUTATTR varexpression AT varexpression, varexpression
* GETATTR (varexpression, varexpression)
* ATTRVAL (expression COMMA expression COMMA expression COMMA expression)

1.1 Requirements and Installation

These are the external requirements for the tool:

- Python 3.11 or higher
- SjAsmPlus 1.20.3 or higher
- ZIP file decompressor

If you upgrade from an older version, it is recommended NOT to overwrite it. It is better to rename the directory of the old version, unzip the new version, copy your adventure files to the new version and configure the `make_adv` script again for each case.

Here are the more detailed instructions for each operating system:

1.1.1 Windows Installation

Installation is easy, just unzip the corresponding ZIP file downloaded from the Releases section of the repository. In this case, everything is included in the package. Windows 10 or higher, 64-bit version is required (32-bit not supported).

1.1.2 Linux, BSDs and UNIX-like Installation

For these systems, the requirements are:

- GNU/Linux / Unix / macOS / BSD with BASH-compatible shell.

- Python 3.11 or higher
- All common system programs (grep, cat, etc…).
- All programs required to compile C programs on the system (libc, libstdc++, g++, GNU make, etc…).
- Autotools (autoconf, automake, aclocals…).
- wget
- git
- libdsk

These requirements are needed to compile `SjAsmPlus` and `TAPTOOLS`. There is no binary distribution of these tools for UNIX-compatible systems, so you need to compile them directly.

Due to the heterodox nature of the different distributions, it is impossible for me to give detailed instructions for installing the requirements in each particular case, so knowledge is required on the part of the user to do so.

The first thing to do is to clone the engine repository to any location you think is convenient and recursively to download the dependencies:

```
git clone --recursive https://github.com/cronomantic/ChooseYourDestiny.git
cd ChooseYourDestiny
```

Now you can proceed to compile **SjASMPlus** with the following commands:

```
cd external/sjasmplus
make clean
make
cd ..
```

And with this we have the dependencies ready. As a last step, we must put the compilation script as executable:

```
cd ../..
chmod a+x make_adv.sh
```

And with this you could already use the tool in Linux.

1.2 CYDC (Compiler)

This program is the compiler that translates the adventure text into a TAP or DSK file. In addition to compiling the adventure into a file that can be interpreted by the engine, it performs a search for the best abbreviations to reduce the size of the text.

```
cydc_cli.py [-h] [-l MIN_LENGTH] [-L MAX_LENGTH] [-s SUPERSET_LIMIT]
              [-T EXPORT-TOKENS_FILE] [-t IMPORT-TOKENS-FILE] [-C EXPORT-CHARSET]
              [-c IMPORT-CHARSET] [-S] [-n NAME] [-img IMAGES_PATH] [-trk TRACKS_PATH]
              [-sfx SFX_ASM_FILE] [-scr LOAD_SCR_FILE] [-v] [-V] [-trim] [-dce] [-code]
              [--no-strict-colons] [--max-errors MAX_ERRORS] [-pause PAUSE_AFTER_LOAD]
              [-wyz] [-il NUM_IMAGE_LINES] [-720]
              {48k,128k,plus3,mld,mld128} input.cyd SJASMPLUS_PATH OUTPUT_PATH
```

- **-h**: Shows the help

- **-l MIN_LENGTH**: The minimum length of the abbreviations to search for (default 3).
- **-L MAX_LENGTH**: The maximum length of the abbreviations to search for (default 30).
- **-s SUPERSET_LIMIT**: Limit for the superset of the search heuristics (default 100).
- **-T EXPORT-TOKENS_FILE**: Export the found abbreviations to the JSON file indicated by the parameter.
- **-t IMPORT-TOKENS-FILE**: Import abbreviations from the indicated file and skip the search for them.
- **-C EXPORT-CHARSET**: Export the 6x8 character set used by default in JSON format.
- **-c IMPORT-CHARSET**: Import the character set to be used in JSON format.
- **-S**: If a compressed text fragment does not fit in a bank, it is split into two between the current bank and the next one with this option activated. Otherwise, the fragment is moved to the next bank.
- **-n NAME**: Name to use for the output file (name for the TAP or DSK file), if not defined it will be the same as the input file.
- **-scr LOAD_SCR_FILE**: Path to an SCR file with the loading screen to use.
- **-img IMAGES_PATH**: Path to the directory with the adventure images.
- **-trk TRACKS_PATH**: Path to the directory with the PT3 AY music files or WyzTracker files.
- **-sfx SFX_ASM_FILE**: Path to an assembler file generated by BeepFx.
- **-v**: Verbose mode, gives more information about the process.
- **-V**: Indicates the version of the program.
- **-trim**: Removes code from commands that are not used in the adventure to make the interpreter smaller.
- **-dce**: Removes bytecode that can never be reached (for example, library routines you include but never call). Reachability follows both jumps and sequential fall-through, so nothing that could run is removed. Especially useful with the bundled libraries.
- **-code**: Shows the generated bytecode.
- **--no-strict-colons**: Allows old syntax without `:` separators between statements on the same line.
- **--max-errors MAX_ERRORS**: Maximum number of parser/preprocessor errors to report before stopping (default 20).
- **-pause**: Number of seconds of pause after finishing the loading process, can be aborted with any keypress.
- **-wyz**: Use WyzTracker music type instead of Vortex Tracker.
- **-il NUM_IMAGE_LINES**: Number of lines to use in image files (default 192).
- **-720**: If using the Plus3 format, a 720 KB disk image will be used instead of the standard 180 KB size.

-**{48k,128k,plus3,mld,mld128}**: Spectrum model to be used: – **48k**: Version for tape in TAP format, does not include the PT3 player and everything is loaded at once. It depends on the size of the available memory. – **128k**: Version for tape in TAP format, everything is loaded at once in the memory banks and depends on the size of the available memory. – **plus3**: This version will generate a DSK file to run on Spectrum+3. Resources are loaded dynamically as needed and depends on the size on disk. – **mld**: Generates an `.MLD` file for the **Dandanator Mini** cartridge in 48K mode. Text, images and bytecode are read directly from the cartridge slots (no RAM banking and no music playback), spanning several Dandanator slots (up to ~256 KB of content). It uses

the cartridge save system. – **mld128**: Generates an `.MLD` file for **Dandanator Mini** targeting Spectrum 128K. Because everything else (text/images/bytecode) is read from the cartridge slots, the RAM banks are reserved for what must live in RAM: **music** (PT3/WyzTracker, which plays from RAM) and the **`DIM` arrays** (which must be writable; they are placed in dedicated RAM banks that never collide with the music). Spans many Dandanator slots (up to ~480 KB of content). Known limitation: if music, arrays and content together force all RAM banks to be used, compilation aborts with a clear error.

- **input.txt**: Input file with the adventure script.
- **SJASMPLUS_PATH**: Path to the SjASMPlus executable.
- **OUTPUT_PATH**: Path where the output files will be stored.

  Note on Dandanator (mld/mld128). The compiler produces an `.MLD` file, which is the game, but it is **not directly loadable**: it must be packed into a 512 KB **Dandanator Mini cartridge ROM**. Use the bundled `mld2rom.py` utility (pure Python; the firmware and menu are built in, no external base ROM is needed):

  ```
  python mld2rom.py -o my_game.rom -a my_game.MLD
  ```

  The `-a` flag enables autoboot (launches the game without going through the menu). The resulting ROM can be flashed onto a real Dandanator Mini or loaded in an emulator that supports it (e.g. **EsPectrum**, or **ZEsarUX** with `--enable-dandanator --dandanator-rom my_game.rom`). The generated ROM is byte-for-byte identical to the one produced by the official dandanator-mini tool.

  As an alternative to the command line, the distribution ships a standalone graphical utility, **`mld2rom_gui.py`** (launchers `mld2rom_gui.cmd` / `mld2rom_gui.sh`), to build the ROM with more options: game name, menu font, background graphic, menu texts, autoboot and border effect.

The compiler is a program written in Python, so it requires the Python environment installed. For convenience, the distribution includes an embedded Python package and a batch script called `cydc.cmd` to launch it from the command line.

The compiler depends on an external program to run, the `SjASMPlus` assembler, so you must specify the path to this program in the input parameters. This program is included in the distribution in the `tools` directory.

You must also specify a destination directory to store the files resulting from the compilation.

Optionally, you can specify the paths to directories containing the compressed images in `scr` format, which will be included on the resulting tape or disk. The same applies to `PT3` files. Both file types must be named with 3-digit numbers, from 0 to 255, such as `000.scr`, `001.scr`, `002.PT3`, and so on. The directories containing both files will be indicated with the `-img` and `-trk‘` parameters, respectively.

The images will be compressed into a `CSC` file. The screens can be full, or the number of horizontal lines can be limited to save memory. It also detects mirrored (symmetrical) images along the vertical axis, so it only stores half of the image. You can even force this behavior and discard the right side of the image to save space. More details on how this works are found in the Images section.

For the music, we can use Vortex Tracker modules (`PT3`) or music created with WyzTracker v2.0

and higher. The behavior is also described in their corresponding sections: Vortex Tracker Tunes and WyzTracker Tunes.

You can add sound effects generated with Shiru's BeepFx application. To use them, export them using the `File->Compile` option in the top menu. In the pop-up window that appears, make sure `Assembly` and `Include Player Code` are selected; the rest of the options are irrelevant. Then save the file somewhere accessible, specifying the path from the command line with the `-sfx` option.

1.3 CYD Character Set Converter

This utility allows you to convert character sets in `.chr`, `.ch8`, `.ch6` and `.ch4` format into a compiler-usable file in JSON format. These formats are editable with ZxPaintbrush.

```
cyd_chr_conv.py [-h] [-w WITDH_LOW] [-W WITDH_UP] [-v] [-V] charset.chr charset.json
```

Supported parameters:

- **-w, –width_low**: Character width (1-8) of the lower character set.
- **-W, –width_high**: Character width (1-8) of the upper character set.
- **-h, –help**: Show help.
- **-v, –verbose**: Verbose mode.
- **-V, –version**: Program version.
- **charset.chr**: Input character set.
- **charset.json**: File with the character set for the compiler.

The default character width is 6 for the lower character set (from 1st to 127th) and 4 for the upper character set (from 145th to 256th), but these values can be changed with the `-w` and `-W` parameters respectively. Note that characters 128 to 144 are special, as they are used for cursors and will always have a width of 8, so the font size will be ignored for those characters. If you want to define a specific width for each character you will have to edit it in the output JSON file. You can find more information in the Character set section.

1.4 Basic Syntax

Commands for the interpreter are enclosed within two pairs of brackets, open and closed respectively. Any text outside of this is considered "printable text", including spaces and line breaks, and will be presented as such by the interpreter. Commands are separated from each other by line breaks or colons if they are on the same line.

Comments within code are delimited by `/*` and `*/`, everything in between is considered a comment.

This is a self-explanatory, abbreviated example of the syntax:

```
This is text [[ INK 6 ]] This is text again but yellow
It's still text [[
/* This is a comment and the following are commands */
WAITKEY
INK 7: PAPER 0
]]
This is text again but white, and watch out for the line break that precedes it!
```

The interpreter runs through the text from the beginning, printing it on the screen if it is "printable text." When a complete word does not fit on the rest of the line, it prints it on the next line. And if it does not fit on the rest of the screen, a wait is generated and the user is asked to press the confirmation key to delete the section of text and continue printing (this last behavior is optional).

When the interpreter detects commands, it executes them sequentially, unless it encounters breaks. Commands allow you to introduce programmable logic into the text to make it dynamic and varied according to certain conditions. The most common and powerful is to ask the player to choose from a series of options (up to a limit of 8 at a time), which can be chosen with the `P` and `Q` keys and selected with `SPACE` or `ENTER`. It supports a Kempston joystick too. Again, this is a self-explanatory example:

```
[[ /* Commands that set screen colors and clear the screen */
PAPER 0 /* Background color to zero (black)*/
INK 7 /* Ink color (white) */
BORDER 0 /* Border color (black) */
CLEAR /* Clear text area (full screen) */
LABEL Location1]]You are in location 1. Where do you want to go?
[[OPTION GOTO Location2]]Go to location 2
[[OPTION GOTO Location3]]Go to location 3
[[CHOOSE]]
[[ LABEL Location2]]You did it!!!
[[ GOTO Final]]
[[ LABEL Location3]]You are dead!!!
[[ GOTO Final]]
[[ LABEL Final : WAITKEY: END ]]
```

The `OPTION GOTO label` command will generate a selection point at the point where the command has been reached. When the `CHOOSE` command is reached, the interpreter will allow the user to choose from one of the option points that have been accumulated on the screen up to that point. A maximum of 32 are allowed.

When choosing an option, the interpreter will jump to the section of text where the corresponding label indicated in the option is found. Labels are declared with the `LABEL identifier` pseudo-command within the code, and when a jump to it is indicated, the interpreter will begin processing from the point where we have declared the label. In the example, if we choose option 1, the interpreter will jump to the point indicated in `LABEL location2`, which will print the text *"You did it!!!"*, and then go to `GOTO Final` which will make an unconditional jump to where `LABEL Final` is defined and ignore everything in between.

Label identifiers only support alphanumeric characters (numbers and letters), must start with a letter and are case-sensitive (upper and lower case are distinguished), i.e. `LABEL Label` is not the same as `LABEL label`. Commands, on the other hand, are not case-sensitive, but for clarity, it is recommended to capitalize them.

A shortened version of labels is also allowed by preceding the label identifier with the character `#`, so that `#MyLabel` is the same as `LABEL MyLabel`. With this, we could rewrite the previous code like this:

```
[[ /* Commands that set screen colors and delete them */
PAPER 0 /* Background color to zero (black) */
INK 7 /* Ink color (white) */
BORDER 0 /* Border color (black) */
CLEAR /* Clear text area (full screen) */
LABEL Location1]]You are in location 1. Where do you want to go?
[[OPTION GOTO Location2]]Go to location 2
[[OPTION GOTO Location3]]Go to location 3
[[CHOOSE]]
[[ #Location2 ]]You did it!!!
[[ GOTO Final ]]
[[ #Location3 ]]You are dead!!!
[[ GOTO Final]]
[[ #Final : WAITKEY: END ]]
```

The available commands are described in their corresponding section.

1.4.1 Strict Colon Mode (Syntax Enforcement)

As of version 1.2.x, the compiler enforces **Strict Colon Mode by default**. This means that when multiple code statements are placed on the same line within `[[ ]]` blocks, they **must** be separated by colons (`:`)

Correct syntax (Strict Colon Mode enabled - DEFAULT):

```
[[ PRINT "Hello" : INK 5 : GOTO Label1 ]]
[[ SET myvar TO 10 : WAITKEY : END ]]
```

Incorrect syntax (missing colons):

```
[[ PRINT "Hello" INK 5 GOTO Label1 ]]  <-- ERROR: Statements must be separated by colons
```

If you need to support old code without colon separators, pass the `--no-strict-colons` flag to the compiler:

```
cydc_cli.py --no-strict-colons 48k input.cyd sjasmplus output
```

Key points: - Line breaks automatically separate statements, so colons are only needed when multiple commands are on the same line - Colons are optional with the `--no-strict-colons` flag for backwards compatibility - This change improves code readability and prevents parsing ambiguities

1.4.2 INCLUDE Directive (Multi-file Projects)

The compiler supports splitting large adventures into multiple source files using `INCLUDE`.

Syntax:

```
[[ INCLUDE "chapters/chapter1.cyd" ]]
```

Rules and behavior: - `INCLUDE` is processed at compile-time by the preprocessor. - The directive must be inside `[[ ]]` code blocks. - Paths are relative to the file containing the directive. - Nested includes are supported up to 20 levels. - Circular includes are detected and reported as errors. - Included files can include other files.

1.5 Bundled libraries

The `lib/` directory ships a set of **libraries written in the CYD language itself** (they do not modify the virtual machine nor add dependencies). They are collections of subroutines that you add to your program with `INCLUDE` and call with `GOSUB`. Each library **skips over itself** (an internal `GOTO` at the top), so you can include it at the start of your script without execution falling into the subroutines: they only run when you call them with `GOSUB`.

```
[[
    INCLUDE "../../lib/math16_32.cyd"
    INCLUDE "../../lib/strings.cyd"
    ... your program ...
]]
```

Each library reserves a block of variables as its workspace. The blocks **do not overlap**, so you can use several at once:

Library	Reserved variables
`math16_32.cyd`	224..254
`strings.cyd`	216..223

1.5.1 `math16_32.cyd` — 16- and 32-bit arithmetic

Provides wide **unsigned** integers (16-bit: 0..65,535; 32-bit: 0..4,294,967,295) with multiplication and division, which CYD's 8-bit variables could not do without overflowing. Operands are loaded into "registers" (groups of 2 or 4 consecutive variables, low byte first): `A` = `mlA0..mlA3`, `B` = `mlB0..mlB3`, `C` = `mlC0..mlC3`. The 16-bit operations use the low 2 bytes of each.

Since CYD literals are 8-bit, wide values are loaded byte by byte with the multiple assignment: `SET mlA0 TO {226, 4}` loads 1250 (= 0x04E2, low byte first).

Routine	Effect
`add16` / `add32`	`A = A + B` (wraps on overflow)
`sub16` / `sub32`	`A = A - B` (wraps; use `cmp` to check first)
`cmp16` / `cmp32`	`mlCmp = 0/1/2` → `A<B` / `A=B` / `A>B`
`shl16` / `shl32`	`A = A << 1`
`shr16` / `shr32`	`A = A >> 1`
`mul32`	`C = A * B` (truncated to 32 bits)
`mul1632`	`C(32) = A(16) * B(16)` — **never overflows** (recommended for scores)
`div32`	`A = A / B` (quotient), `C = A mod B` (remainder)
`print16` / `print32`	prints `A` in decimal (destroys `A`)

To divide by a 16-bit value, load it into `B` with `mlB2 = mlB3 = 0` and use `div32` (the quotient may be 32-bit). The 16-bit tier is ~2–4× faster; avoid `mul`/`div` inside very tight loops. There is a full example in `examples/math_library`.

Example: score = enemies × points + bonus = 1250 × 800 + 234567.

```
[[
    INCLUDE "../../lib/math16_32.cyd"
    SET mlA0 TO {226, 4}       /* A = 1250 */
    SET mlB0 TO {32, 3}        /* B = 800  */
    GOSUB mul1632              /* C = 1000000 (no overflow) */
    SET mlA0 TO {@mlC0, @mlC1, @mlC2, @mlC3}   /* A = C */
    SET mlB0 TO {71, 148, 3, 0}   /* B = 234567 */
    GOSUB add32                /* A = 1234567 */
]]Score: [[ GOSUB print32 ]]
```

1.5.2 `strings.cyd` — text strings

Keyboard input, printing and handling of character strings stored in consecutive variables (using `[@ptr]` indirection), terminated by `0`. Before calling, set the registers: `stBase` (index of the first variable of the buffer), `stLen` (capacity in characters) and, for copy or compare, `stB2` (index of the second buffer).

Routine	Effect
`strClear`	zero-fills the buffer
`strLen`	counts characters up to the 0 or the end → `stRes`
`strPrint`	prints the buffer (`CHAR`) up to the 0 or the end
`strInput`	reads a string from the keyboard (cursor, `ENTER` ends, `DELETE` erases)
`strCopy`	copies the `stBase` buffer → `stB2` (`stLen` bytes)
`strCmp`	compares `stBase` with `stB2` → `stRes = 0/1/2` (less/equal/greater)

Example: ask for a name and greet.

```
[[
    INCLUDE "../../lib/strings.cyd"
    SET stBase TO 0 : SET stLen TO 16
    GOSUB strInput
]]Hello, [[ GOSUB strPrint ]]!
```

There is a full example in `examples/strings_library`.

1.6 Native routines (IMPORT / CALL)

For jobs the virtual machine cannot do — reading or writing a hardware port, touching an arbitrary memory address, or a tight piece of number-crunching that needs to be fast — you can write a small routine in **Z80 assembly** and call it from your script.

Advanced, use at your own risk. This runs your own native code with no sand-

box: a bug in the routine can hang or crash the machine, exactly like the BeepFX or WyzTracker code you already trust. If you are not comfortable with Z80 assembly, you do not need this feature.

Two keywords are involved:

```
[[
    IMPORT beeper FROM "routines/beeper.asm"   ; declare (compile-time)
    ...
    CALL beeper                                ; run it (at runtime)
]]
```

- **`IMPORT name FROM "file.asm"`** registers a native routine under `name`. It is a compile-time declaration (like `DECLARE` or `CONST`, **not** like `INCLUDE`, which includes CYD source). A relative path is resolved relative to the folder of your `.cyd` script.
- **`CALL name`** runs the routine. Think of it as the native counterpart of `GOSUB`: `GOSUB label` calls a subroutine written in CYD, `CALL name` calls one written in Z80.

1.6.1 Writing the routine

You write **only the body** of the routine — no `ORG`, no directives, no `SAVEBIN`. The compiler frames it, assembles it **in isolation** (so a mistake in your file produces a clean error attributable to *your* file, without breaking the engine build) and places it in memory for you, re-assembling it at its final address so absolute addresses just work.

The contract (ABI) is deliberately small:

- The routine is entered with **`DE = FLAGS`**, the base address of the 256-byte variable array. Every CYD variable `n` is the byte at `FLAGS + n`.
- **Inputs and outputs travel through variables.** The script puts arguments in variables with `SET`, calls the routine, and reads results back with `@var`. The routine reads and writes them at `FLAGS + n`. There is no other calling convention to remember.
- It must end with **`RET`** and must not jump into the engine's internals on its own. It may freely use `AF`/`BC`/`DE`/`HL`/`IX`/`IY` (the engine saves and restores `IX`/`IY`, which it relies on internally). For what you *do* need — reaching arrays across banks, or calling another native routine — there are **resident services** documented below (`CYD_PEEK`/`CYD_POKE`, `CYD_ARR_MAP`/`CYD_ARR_FLUSH`, `CYD_CALL`).
- It **must fit in one 16 KB bank** and must not leave the hardware in a state that breaks the engine on return (if it changes paging via `$7FFD`, it must restore it).

1.6.2 Example

```
[[
    IMPORT peek FROM "peek.asm"
    DECLARE 0 as addr_lo
    DECLARE 1 as addr_hi
    DECLARE 2 as result
    SET addr_lo TO 0 : SET addr_hi TO 0   ; address 0000h
    CALL peek                             ; result <- byte at that address
]]
```

with `peek.asm` (you write just this):

```
    ld a, (de)          ; DE = FLAGS -> A = FLAGS+0 = address low byte
    ld l, a
    inc de
    ld a, (de)          ; A = FLAGS+1 = address high byte
    ld h, a             ; HL = the address to read
    inc de              ; DE -> FLAGS+2
    ld a, (hl)          ; A = the byte at that address
    ld (de), a          ; FLAGS+2 = result
    ret
```

There is a full, runnable example in `examples/import_demo`.

1.6.3 Writing the routine inline (`ASM … ENDASM`)

You don't always want a separate file. You can write the body **directly in the `.cyd`** between `ASM name` and `ENDASM`, and call it with `CALL` as usual:

```
[[
    ASM put42
        ld a, 42
        ld (de), a          ; DE = FLAGS -> FLAGS+0 = 42
        ret
    ENDASM
    CALL put42
]]
```

- The body is taken **verbatim**: `;` comments, labels, `0x…` numbers… all go to sjasmplus without CYD interpreting them. You may **indent the block** however you like to line it up with the `[[ ]]`: CYD moves label (and `EQU`) lines to column 0 for you; just keep the instructions indented more than the labels.
- The **same ABI** applies as with `IMPORT` (entered with `DE = FLAGS`, ends in `RET`) and the same `CALL`. `ASM … ENDASM` is just an `IMPORT` with the body inline. If sjasmplus reports an error, CYD remaps it to your **`.cyd` line**.

1.6.4 Blocks with several entry points (`EXPORTS`)

A block can expose **several routines** that share private helpers and call each other with a plain `call` (they live in the same bank). Declare them with `EXPORTS`:

```
[[
    ASM matrix EXPORTS put_a, put_b
    put_a:
        ld a, 42
        jr store
    put_b:
        inc de
        ld a, 99
    store:
        ld (de), a          ; private, shared helper
```

```
        ret
    ENDASM
    CALL put_a              ; FLAGS+0 = 42
    CALL put_b              ; FLAGS+1 = 99
]]
```

Each `EXPORTS` name is a `CALL` target. The block is assembled and placed as **one unit** (the natural unit of a "library").

1.6.5 Reaching the engine's data

When it assembles your routine, CYD injects the addresses of the resident structures by name, so you don't have to guess them:

Symbol	What it is
`FLAGS`	base of the 256-variable array (same as `DE` on entry)
`SCREEN_BUFFER_PXL` / `SCREEN_BUFFER_ATT`	the engine's image buffer (pixels / attributes)
`VIDEO_PXL` / `VIDEO_ATT`	the physical Spectrum screen (`$4000` / `$5800`)

So, for instance, `ld hl, SCREEN_BUFFER_PXL` gives you the image buffer directly.

1.6.5.1 Arrays

Your script's arrays (`DIM`) are reachable too. For each array `<name>` CYD injects:

Symbol	What it is
`ARR_<name>`	address of element 0
`ARR_<name>_LEN`	element count
`ARR_<name>_BANK`	physical bank it lives in, or `$FF` if resident

On 128K/+3 an array may live in a **paged bank** that your routine can't map without evicting itself. So access goes through **resident services** that do the paging for you (on 48K, or when the bank is `$FF`, they are direct access). All of them preserve `BC/DE/HL/IX/IY` (except the result):

- **`CYD_PEEK`** — read a byte. In: `A = bank`, `HL = address`; out: `A = byte`.
- **`CYD_POKE`** — write a byte. In: `A = bank`, `HL = address`, `E = byte`.
- **`CYD_ARR_MAP`** — copy the whole array to a resident buffer and return a direct pointer so you can work without paging. In: `A = bank`, `HL = ARR_<n>`, `BC = ARR_<n>_LEN`; out: `HL = working pointer`.
- **`CYD_ARR_FLUSH`** — write that buffer back to the array. Same input parameters. (Only one array mapped at a time.)

Example — increment element 3 of `inv`:

```
[[
    DIM inv(4) = {10, 20, 30, 40}
    ASM inc3
        ld a, ARR_inv_BANK
```

```
        ld hl, ARR_inv + 3
        call CYD_PEEK       ; A = inv[3]
        inc a
        ld e, a
        ld a, ARR_inv_BANK
        ld hl, ARR_inv + 3
        call CYD_POKE       ; inv[3] = inv[3] + 1
        ret
    ENDASM
    CALL inc3
]]
```

If you are going to touch many elements, `CYD_ARR_MAP` + `CYD_ARR_FLUSH` is faster: map once, walk the pointer directly, and flush at the end.

1.6.6 Calling another native routine (`USES` + `CYD_CALL`)

Two routines in the **same block** (`EXPORTS`) call each other with a plain `call`. To call a routine in **another block** (which on 128K/+3 may be in another bank), use the resident trampoline `CYD_CALL` and declare whom you call with `USES`:

```
[[
    ASM greeter EXPORTS greet
    greet:
        ld a, 55
        ld (FLAGS+1), a
        ret
    ENDASM
    ASM main USES greet
        ld a, 11
        ld (FLAGS), a
        ld a, RT_greet      ; index injected by USES
        call CYD_CALL       ; calls 'greet', whatever bank it is in
        ret
    ENDASM
    CALL main               ; FLAGS = [11, 55]
]]
```

- `USES a, b, …` declares the callees; for each one CYD injects an `RT_<name>` index.
- `ld a, RT_<name> : call CYD_CALL` runs that callee. It is entered with `DE = FLAGS` (like a script `CALL`) and CYD does the paging for you.

1.6.7 Calling engine services (`CYD_SYSCALL`)

Some things are done by the engine, not the virtual machine: **printing a character** or **reading the keyboard**. Your routine can ask for them through a single entry point, `CYD_SYSCALL`, passing the **service number** in `A` (the `SVC_*` constants CYD injects) and the arguments in registers:

```
[[
    ASM greet
        ld e, 72            ; code for 'H'
        ld a, SVC_PRINT_CHAR
        call CYD_SYSCALL    ; prints 'H' at the cursor
        ret
    ENDASM
    CALL greet
]]
```

Available services (a small set to start with; it will grow):

Service	In / out
`SVC_PRINT_CHAR`	`E` = character — prints it at the cursor (advances it, honours ink/paper/window)
`SVC_WAIT_KEY`	waits for a key → `A` = code
`SVC_INKEY`	reads a key without waiting → `A` = code (`0` if none)

The service numbers are a **stable contract**: even if the interpreter is rewritten internally, your routines keep working without recompiling. Treat `CYD_SYSCALL` like any `call` (it may clobber registers); in particular, do not touch `IY` before it.

1.6.8 Unused routines

An `IMPORT/ASM` routine that **nobody calls** is not assembled and takes no memory: CYD drops it automatically. So you can keep a "library" of routines and pay only for the ones you actually use. (The resident services above — the array broker, `CYD_CALL`, `CYD_SYSCALL` — are likewise removed from the engine if no routine uses them.)

There is an example with `ASM`, `EXPORTS`, arrays and `CYD_CALL` in `examples/inline_asm`.

1.6.9 Supported targets

Native routines work on **48K, 128K, +3 and mld128**. On the 128K, +3 and mld128 the routine is placed in a paged RAM bank and the engine pages it in around the `CALL` automatically. (The strict 48K **mld** target —a single bank— does not support them and compilation aborts with a clear error.)

1.7 Variables and Numeric Expressions

Numeric constant values can be expressed in base 10 by default, in hexadecimal with the prefix `0x`, or in binary with the prefix `0b`. For example, `240` would be decimal, `0xF0` in hexadecimal, and `0b11110000` in binary.

There are 256 one-byte containers (from 0 to 255) available to the programmer to store values, perform operations, and compare them. They constitute the state of the program. From this point on, they may be called variables, flags, or "variables" interchangeably throughout this document.

To refer to a variable in a numeric expression, the number must be preceded by the `@` character. But it is possible to give a meaningful name to variables using the `DECLARE` command as follows:

```
[[
DECLARE 7 AS InkColor
INK @InkColor
]]
```

It should be noted that a variable cannot be declared twice, nor can there be labels and variables with the same names, but synonyms are allowed as follows:

```
[[
DECLARE 10 AS SomeName
DECLARE 10 AS OtherName
]]
```

Thus, both *SomeName* and *OtherName* will serve to identify variable 10. Keep in mind that despite having different names, they are the same variable.

To assign values to a variable, we use the command `SET varId TO varexpression`, as follows:

```
[[
SET 0 TO 1 /* Set the variable zero to 1 */
SET variable TO 2 /* Set the variable named variable to 2 */
SET variable2 TO @variable + 2 /* Set the variable named variable2 to two plus the value of the variabl
SET variable2 TO @variable2 - (@variable + 2) /* Allows parentheses */
]]
```

We also now have the following synonym with `LET varId = varexpression`, so the previous examples could be rewritten like this:

```
[[
LET 0 = 1 /* Set the variable zero to 1 */
LET variable = 2 /* Set the variable named variable to 2 */
LET variable2 = @variable + 2 /* We set the variable called variable2 to two plus the value of the vari
LET variable2 = @variable2 - (@variable + 2) /* Allows parentheses */
]]
```

As you can see, a variable can be assigned the value of a mathematical expression composed of numbers, functions and variables. As already indicated, variables within a numeric expression, that is, the right side of an assignment **must always be preceded by the `@` character**.

The available operands are:

- Addition: `SET variable TO @variable + 2`
- Subtraction: `SET variable TO @variable - 2`
- Binary "AND": `SET variable TO @variable & 2`
- Binary "OR": `SET variable TO @variable | 2`
- Binary "NOT" or bit complement: `SET variable TO !@variable`
- Bit shift left: `SET variable TO @variable << 2`
- Bit shift right: `SET variable TO @variable >> 2`

Additionally, `LET` supports shorthand operators to modify a variable in place:

- Increment: `LET variable += 2` (equivalent to `LET variable = @variable + 2`)
- Decrement: `LET variable -= 2` (equivalent to `LET variable = @variable - 2`)

These also work with indirection and arrays: `LET [variable] += 1` and `LET myArray(0) += 1`.

The result of a numeric expression cannot be greater than 255 (1 byte) or less than zero (negative numbers are not supported). If both limits are exceeded when performing the operations, the result will be adjusted to the corresponding limit, that is, if an addition exceeds 255, it will be adjusted to 255 and a subtraction that gives a result less than zero will be set to zero.

One thing to note is that binary operators are not the same as the logical operators of conditional expressions described below. An **&** is not the same as an **AND**. Binary operators perform the corresponding operations on the bits of the variable.

Most commands accept numeric expressions in their parameters, for example:

```
[[
INK 7
INK @7
]]
```

The first command will set the text color to white (color 7), while the second will set the text color to the value contained in the variable number 7. Combining all of the above, you could do:

```
[[
INK 3+4
INK @Var-1
]]
```

Check the commands reference to find out which commands accept them.

Finally, there are a number of special commands called **functions**. These commands cannot be used on their own, but must be used within a numeric expression, since they return a value to be used within it.

- **RANDOM(expression)**: Returns a random number between 0 and the value specified in **expression**.
- **RANDOM()**: Returns a random number between 0 and 255. This is equivalent to `RANDOM(255)`.
- **RANDOM(expression,expression)**: Returns a random number between the value indicated in the first parameter and the second, both inclusive.
- **INKEY()** Waits until a key is pressed and returns the code of the key pressed. (Only for keyboard).
- **KEMPSTON()** Returns the buttons pressed on a Kempston joystick connected to the Spectrum.
- **MIN(varexpression,varexpression)**: Limits the value of the first parameter to the minimum indicated by the second. That is, if parameter 1 is less than parameter 2, parameter 2 is returned, otherwise 1 is returned.

- **MAX(varexpression,varexpression)**: Limits the value of the first parameter to the maximum indicated by the second. That is, if parameter 1 is greater than parameter 2, parameter 2 is returned, otherwise 1 is returned.
- **POSY()**: Returns the current row the cursor is on in 8x8 coordinates.
- **POSX()**: Returns the current column the cursor is on in 8x8 coordinates (Due to the nature of variable width font, the returned value will be the 8x8 column where the cursor is currently located).
- **LASTPOS(ID)**: Returns the index of the last position in the given array.

So, for example `SET variable_no TO RANDOM(6)`, assigns to the variable *variable_no* a random number from 0 to 6. And in the same way, we can operate with them: `SET variable_no TO RANDOM(6) + @var + 1`

1.8 Flow control and conditional expressions

To control the flow of execution, we have different commands, which I will now describe:

- **GOTO ID**

Jumps to the label *ID*.

- **GOSUB ID**

Subroutine jump, jumps to the label *ID*, but returns execution to the next instruction to the `GOSUB` as soon as it finds a `RETURN` command.

- **RETURN**

Returns to the point after the subroutine call, see `GOSUB`

- **IF condexpression THEN ... ENDIF**

If the conditional expression *condexpression* is true, the text is printed and the commands from `THEN` to `ENDIF` are executed.

- **IF condexpression THEN ... ELSE ... ENDIF**

If the conditional expression *condexpression* is true, the text is printed and the commands from `THEN` to `ELSE` are executed. Otherwise, the text is printed and the commands from `ELSE` to `ENDIF` are executed

- **IF condexpression THEN ... ELSEIF condexpression THEN [... ELSEIF condexpression THEN] ... ELSE ... ENDIF**

If the conditional expression *condexpression1* evaluates to true, the text is printed and the commands from `THEN` to `ELSE` are executed. Otherwise, the conditional expression *condexpression2* is checked, in which case the text is printed and the commands from `ELSEIF` to the `ELSE` or another `ELSEIF` are executed. Multiple `ELSEIF`s can be chained together until a final `ELSE` clause is reached, which is executed if none of the previous conditions have been met.

- **WHILE (condexpression) ... WEND**

The text and commands up to `WEND` are repeated repeatedly as long as the condition *condexpression* evaluates to true. Evaluation is performed at the beginning of each iteration.

- **DO ... UNTIL (condexpression)**

The text and commands from `DO` to `UNTIL` are repeated repeatedly as long as the *condexpression* condition evaluates to false. Evaluation is performed at the end of each iteration.

- **FOR variable = start TO end [STEP step] ... NEXT [variable]**

Counted loop: the text and commands up to `NEXT` are repeated, running *variable* from *start* to *end*. `STEP` is **optional** and defaults to `1`; it may be negative (`STEP -2`) to count down. The step must be a **constant** (not a variable or a `CONST`), because the compiler needs to know the loop direction. The counter variable is written **bare** (as in `SET`/`LET`); inside the body you read its value with `@variable`. `NEXT` may optionally carry the variable name (`NEXT i`), which must match. The *start* and *end* limits may be expressions (even variables); *end* is re-evaluated on every iteration. If the counter has already passed the limit at the start, the loop runs **zero times** (BASIC semantics). Example:

```
[[
DECLARE 0 AS i
FOR i = 1 TO 5           /* i = 1, 2, 3, 4, 5 */
  PRINT @i
NEXT
FOR i = 10 TO 0 STEP -2  /* count down: 10, 8, 6, 4, 2, 0 */
  PRINT @i
NEXT
]]
```

The counter is a **byte** (0..255). The loop is internally protected against byte overflow, so `FOR i = 0 TO 255` and `FOR i = 10 TO 0 STEP -1` work correctly (they do not become infinite loops). Do keep in mind, though, that the limits must fit in a byte.

As you can see, many of these functions are executed depending on whether a condition is met. For example:

```
[[IF @var = 0 THEN GOTO jump ENDIF]]
```

For the jump to be executed, the variable *var* must be equal to zero. This is what is called a conditional expression.

A condition is always a comparison between two numeric expressions:

- Equal to: `IF @variable = 0 THEN GOTO jump ENDIF`
- Greater than: `IF @variable > 0 THEN GOTO jump ENDIF`
- Less than: `IF @variable2 < @variable THEN GOTO jump ENDIF`
- Not equal to: `IF @variable <> 0 THEN GOTO jump ENDIF`
- Greater than or equal to: `IF @variable <= 0 THEN GOTO jump ENDIF`
- Less than or equal to: `IF @variable >= 0 THEN GOTO jump ENDIF`

In addition, conditions can be combined into conditional expressions using logical operations. For example:

```
[[IF (@variable = 0 AND @variable2 = 1) AND NOT @variable3 = 1 THEN GOTO jump ENDIF]]
```

- Cond1 **AND** Cond2: True if both Cond1 and Cond2 are true.
- Cond1 **OR** Cond2: True if either Cond1 or Cond2 is true.
- **NOT** Cond1: True if the condition Cond1 is false.

Again, remember that the logical operators of conditional expressions **are not the same as binary operators**.

Also, for those with advanced programming knowledge, conditions are not "short-circuiting", that is, all logical operations and comparisons are evaluated until the end.

1.9 Assignments and indirection

We have already seen assignments and numerical expressions in their section, but we are going to explain in more detail how they work with indirection, which deserves a separate section.

Indirection is done by putting a numerical expression in single brackets, for example `[@var+1]`. What we are trying to say is that **we are going to refer to the variable or variables with the number calculated by the numerical expression enclosed in brackets**. Let's see it with examples:

- `SET var to [2+2]`: indicates that we are going to assign to *var* the content of the variable 2+2, that is, 4. This is equivalent to `SET var TO @4`, but it allows us to strengthen the concept.
- `SET var TO [@index]`: this is where the utility really comes into play, since we are indicating that we are saving in *var* the content of the variable whose number corresponds to the content of the variable *index*. That is, we can dynamically change the variable to which we make the assignment.
- `SET var TO [@index+2]`: indirection supports any numeric expression, this means that we save in *var* the content of the variable that corresponds to the content of *index* plus two.
- `SET [var] TO 0`: Indirection can also be used on the right-hand side, and in this case, numeric expressions are not supported, only the number or the variable identifier if it has been declared. This means "assign zero to the variable whose index corresponds to the content of the variable *var*.
- `INK [@var]`: Indirection is also supported by those commands that admit numeric expressions.
- `SET [@@var] TO 0`: For completeness, since we operate with pointers, using `@@` we have the complementary operation to indirection, that is, it returns the index number of a given variable, so in the example, the operation would be equivalent to `SET var TO 0`.

If we use numeric indicators with variables (e.g. `@0`) it is not very useful. Its real utility lies in using it with variable identifiers, such that if we have:

```
[[
DECLARE 20 AS var : DECLARE 10 AS index: SET index AS @@var : SET [index] AS 0
]]
```

We see that this allows us to initialize variables that we are going to use. to be used for indirection with the identifier of another variable.

Indirection is a powerful tool that allows us to implement arrays in variables, but be careful, we only have 256! Also, we must not confuse them with the arrays described in the next section, which are of a different nature.

Finally, we can perform multiple assignments using the following construction:

```
[[
DECLARE 10 AS var
SET var AS {0, 1, 2, 3}
]]
```

This would be equivalent to:

```
[[
SET 10 AS 0
SET 11 AS 1
SET 12 AS 2
SET 13 AS 3
]]
```

That is, we can consecutively assign a sequence of values to a sequence of variables, starting with the variable indicated on the left side. This allows multiple values to be assigned consecutively.

1.10 Constants

During development, we may need to have values with a meaningful name. Just like with variables, we can name them with `CONST name = value`, and then use `name` instead. For example:

```
[[
CONST maxBalas = 10 /* we declare the constant maxBalas with value 10 */
PRINT maxBalas /* we print the value 10 */
]]
```

Unlike variables, we can use as many constants as we want, since during the compilation process they will be replaced by their corresponding value.

The name of the constant cannot match that of another variable, label or any of the commands.

1.11 Arrays or “sequences”

In the previous section we have described the use of indirection to use variables as an array, but `CYD` also has an alternative way of creating data arrays using the `DIM` command, such that with `DIM name(size)` we declare an array with name `name` and size `size`. The size of an array cannot be greater than 256 nor be zero. We access each of the elements of the array with the nomenclature `name(pos)`, where `pos` is the position number of the element to be accessed within the array, starting from zero. Let’s see it with examples:

```
[[
DIM myArray(5) /* We declare an array of 5 elements from 0 to 4 */
LET myArray(3) = 1 /* We assign 1 to position 3 of the array, which would be the fourth */
PRINT myArray(3) /* We print the value stored in position 3 */
]]
```

If we try to access a position outside the range, the engine will fail and give an error. To find out which is the last accessible position of the array, we have the `LASTPOS` function:

```
[[
DIM myArray(5) /* We declare an array of 5 elements */
PRINT LASTPOS(myArray) /* It would return 4 */
]]
```

Assigning initial values to an array can be done in the following way: `DIM array_name(size) = {value1, value2, ...}`. Following the previous example:

```
[[
DIM myArray(5) = {1, 2, 3, 4, 5} /* We assign values to the array */
PRINT myArray(1) /* This would print 2 */
]]
```

Uninitialized positions will always have a value of 0 by default, so we can have an array with more positions than values. The opposite would give us an error:

```
[[
DIM myArray(5) = {1, 2, 3} /* This is allowed */
DIM myArray(3) = {1, 2, 3, 4, 5} /* This would give an error */
]]
```

Finally, and as a convenience, we can leave the number of positions of the array empty when declaring it with values. The compiler will create an array with as many positions as there are elements in the initialization:

```
[[
DIM myArray() = {1, 2, 3} /* This is allowed, myArray would have 3 positions */
DIM myArray() /* This would give an error */
]]
```

This type of arrays have a series of limitations that must be taken into account when using them. The most obvious one is that they cannot have more than 256 elements and are one-dimensional and cannot be resized.

Another limitation is that you cannot save or retrieve data from these arrays using `SAVE` and `LOAD` directly. The only option is to move the array data to or from variables and perform save or retrieve operations on them.

Finally, if you modify the data in an array, you cannot retrieve the original values unless you make a copy to another array first.

1.12 Immutable data (`DATA` / `READ` / `RESTORE` / `DATAEND()`)

When you need a **read-only** block of data (tables, maps, byte-by-byte packed text, sequences…), `CYD` offers the `DATA` mechanism, analogous to BASIC's. Unlike `DIM` arrays —which are writable and limited to 256 elements—, `DATA` is **immutable** and forms a **single global stream** that can be up to **16 KB** (over 16000 elements). Since it is never modified, it takes **0 bytes of RAM on Dandanator/MLD** (it is read straight from flash).

Each `DATA` statement appends its bytes, in source order, to that global stream. They are read sequentially with `READ`, which uses the same bare destination as `SET`/`LET`:

```
[[
  DECLARE 0 AS v

  DATA 10, 20, 30      /* these five values form a single stream:  */
  DATA 40, 50          /* 10, 20, 30, 40, 50                       */

  READ v               /* v = 10, and the cursor advances     */
  READ [v]             /* indirect destination (index in v), like LET [v] */
]]
```

There is a **single global cursor** that starts at the beginning of the stream. `RESTORE` rewinds it to the start, and `RESTORE label` places it at the **first `DATA` appearing after that label** (same semantics as BASIC's `RESTORE line`; it reuses `LABEL`s):

```
[[
  DECLARE 0 AS v

  DATA 11, 22, 33
  LABEL second
  DATA 44, 55

  RESTORE second       /* the cursor is placed at '44' */
  READ v               /* v = 44                       */
]]
```

The `DATAEND()` function returns `1` if the cursor has reached the end of the stream and `0` otherwise. It is useful to walk all the data with a loop (note it is a function, with parentheses, and to use it as a condition you compare it, e.g. `DATAEND() = 0`):

```
[[
  DECLARE 0 AS v

  DATA 65, 66, 67

  WHILE (DATAEND() = 0)  /* while we have not reached the end */
    READ v
    CHAR v               /* prints the character A, B, C      */
  WEND
]]
```

Things to keep in mind:

- `DATA` statements **are not executed**: they can be placed anywhere in the script (the compiler gathers them all and removes them from the code stream). Grouping them helps readability.
- **Endless tape**: reading past the end automatically rewinds the cursor to the start of the stream (it is not an error). Use `DATAEND()` if you want to detect the end and stop.
- Values are byte constants (they accept the same expressions and named constants as a `DIM` initializer).
- If a program does not use `DATA`, the corresponding opcodes are stripped from the interpreter (zero cost).

1.13 Wide constants (`WORD` / `DWORD` / strings)

`CYD` works with bytes, but you often need to place **16-bit** (0..65535) or **32-bit** (compatible with the `math16_32` library) values in memory, or the character codes of a **string**. That is what wide constants are for: values the compiler expands into a **fixed sequence of consecutive bytes in *little-endian* order** (low byte first). Everything happens at compile time: they add **no runtime**.

They can be used in three places: in `DATA`, in a `DIM` initializer and in `LET`/`SET` of a variable.

- **Auto-detection by magnitude** (only in `DATA` and `DIM`): a value from 0 to 255 takes 1 byte, from 256 to 65535 takes 2 bytes, and 65536 upwards takes 4 bytes, automatically.
- **`WORD` and `DWORD` markers**: force 2 or 4 bytes even if the value fits in fewer (`WORD 5` = 2 bytes). `WORD` does not accept values larger than 16 bits and `DWORD` not larger than 32 bits.
- **Strings** `"..."`: expand to their character codes (no terminator; you know the length because you wrote it).

```
[[
  DATA WORD 1000, 5, "HI"          /* 2 bytes + 1 byte + 2 bytes */
  DIM table() = { DWORD 100000, 42 }   /* 4 bytes + 1 byte -> 5-byte array */
]]
```

Arrays stay **byte by byte** (*byte-flat*): there is no "wide" access to an array; you recompose the value yourself by reading the bytes you need (with `math16_32` if you are going to operate on it).

In `LET`/`SET`, the width is **always** set by the keyword (`WORD`/`DWORD`) or the string; the bytes are written into the destination variable and the following consecutive ones. There is **no auto-detection** here (the number of variables used must be known at compile time, so a `LET v = 1000` without a marker would still be 1 byte and would error for not fitting):

```
[[
  DECLARE 0 AS score      /* uses variables 0 and 1 */
  DECLARE 2 AS name       /* uses variables 2 and 3 */
  LET score = WORD 1000   /* var 0 = low byte (232), var 1 = high byte (3) */
  LET name = "AB"         /* var 2 = code of 'A', var 3 = code of 'B'       */
]]
```

Only a **direct** destination is allowed (known consecutive indices): the `LET [v]` indirection cannot be used with wide constants.

1.14 Handy literals (character, ranges, repetition)

To avoid memorizing codes or writing long sequences by hand, `CYD` offers several literals the compiler expands at compile time (0 runtime):

- **Character literal `'A'`** -> the **glyph code** of the character. Valid anywhere a number goes (`DATA`, `DIM`, `LET`, expressions, `CASE`…). E.g. `LET key = 'A'` is the same as `LET key = 65`.
- **Ranges `{a..b}`** -> the byte sequence `a, a+1, ..., b` (or descending if `a > b`). The bounds are byte constants and accept characters: `'A'..'Z'`. E.g. `DIM count() = { 1..8 }`.
- **Repetition `{ v REPEAT n }`** -> `n` copies of `v` (which may be a value, a wide constant, a string or a range). E.g. `DIM buffer() = { 0 REPEAT 16 }` (sixteen zeros).

Ranges and repetition only make sense in **list** contexts (`DATA`, `DIM` initializers), and combine with each other: `DATA 255 REPEAT 4, 1..3, 'A'`.

1.15 List of commands

Before describing the commands, let's briefly discuss the legend used:

- **ID** is an identifier and is a string of numbers, letters or underscores. Accented letters and ñ are not allowed.
- **expression** is a numeric expression without variables. It is a decimal or hexadecimal number, but simple arithmetic operations are also allowed and will be calculated at compile time.
- **varexpression**, numeric expression as described in its corresponding section. It can contain variables with `@` and indirections.
- **condexpression**, conditional expression as described in its corresponding section. They consist of one or several comparisons with logical operations between them.
- **varID**, variable identifier, can be an *expression* if we use its numeric index or an *ID* if we have declared it like this with `DECLARE`.

Here is the full list of commands:

1.15.1 INCLUDE "path/to/file.cyd"

Compile-time directive that inserts another `.cyd` file in the current source. It must be used inside `[[ ]]` blocks. It is resolved before parsing, supports nested includes, and prevents circular references.

1.15.2 LABEL ID

Declare the label labelId at this point. All jumps with reference to this label will direct execution to this point.

1.15.3 #ID

Shortened version for declaring the label *ID*, such that `#LabelId` is the same as `LABEL LabelId`.

1.15.4 DECLARE expression AS ID

Declare the identifier *ID* as a symbol representing the variable *expression* instead.

1.15.5 CONST ID = expression

Declare the constant *ID* with the value of *expression*.

1.15.6 DIM ID(expression)

Create an array named *ID* with as many elements as indicated by *expression*. The number of elements cannot exceed 256.

1.15.7 DIM ID(expression) = {expression, expression...}

Same as DIM ID(expression), but assigning comma-separated values. Elements may be byte constants, **wide constants** (`WORD`/`DWORD` or auto-detected) or strings; the array stays byte by byte (see Wide constants).

1.15.8 DATA expression, expression...

Appends the values to the global read-only immutable data stream, in source order. All `DATA` statements in the script form a single stream (16 KB max). They are not executed; the compiler extracts them from the code. Each element may be a byte constant, a **wide constant** (`WORD`/`DWORD` or auto-detected by magnitude) or a string; see Wide constants.

1.15.9 READ varID

Reads the next byte of the data stream into variable *varID* and advances the cursor.

1.15.10 READ [varID]

Same as READ varID, but the destination is the variable whose index matches the content of *varID* (indirect destination).

1.15.11 RESTORE

Rewinds the data stream cursor to the beginning.

1.15.12 RESTORE ID

Places the data stream cursor at the first `DATA` appearing after the *ID* label in the source.

1.15.13 DATAEND()

Returns `1` if the data stream cursor has reached the end, `0` otherwise.

1.15.14 GOTO ID

Jump to the *ID* label.

1.15.15 GOSUB ID

Subroutine jump, jumps to the *ID* label, but returns to the next command when it encounters a `RETURN` command.

1.15.16 RETURN

Returns to the point after the subroutine call, see `GOSUB`

1.15.17 IF condexpression THEN ... ENDIF

If the conditional expression *condexpression* is true, the text is printed and the commands from `THEN` to `ENDIF` are executed.

1.15.18 IF condexpression THEN ... ELSE ... ENDIF

If the conditional expression *condexpression* is true, the text is printed and the commands from `THEN` to `ELSE` are executed. Otherwise, the text is printed and the commands from `ELSE` to `ENDIF` are executed.

1.15.19 IF condexpression1 THEN ... ELSEIF condexpression2 THEN ... ELSE ... ENDIF

If the conditional expression *condexpression1* is true, the text is printed and the commands from `THEN` to `ELSE` are executed. Otherwise, the conditional expression *condexpression2* is checked, in which case the text is printed and the commands from `ELSEIF` to the `ELSE` or another `ELSEIF` are executed. Several `ELSEIF`s can be chained together until a final `ELSE` clause is reached, which is executed if none of the previous conditions have been met.

1.15.20 WHILE (condexpression) ... WEND

Text and commands up to `WEND` are repeated repeatedly as long as the *condexpression* condition evaluates to true. Evaluation is performed at the beginning of each iteration.

1.15.21 DO ... UNTIL (condexpression)

Text and commands from `DO` to `UNTIL` are repeated repeatedly as long as the *condexpression* condition evaluates to false. Evaluation is performed at the end of each iteration.

1.15.22 SELECT varexpression ... CASE value[,value...] ... [CASE ELSE ...] ENDSELECT

Multi-branch: evaluates *varexpression* and runs the `CASE` branch whose value matches (`CASE 1, 2, 3` = several values for the same branch). `CASE ELSE` (optional) is the default branch. **No fall-through**: when a branch ends it jumps to the end (BASIC semantics). The subject is re-evaluated on each comparison (like the FOR limit), so keep it a variable or a side-effect-free expression. See flow control.

1.15.23 ENUM [name] { A, B=value, C, ... }

Declares sequential constants (like several `CONST`): auto-increment from 0, or from the last explicit value (C style). `ENUM { NORTH, SOUTH, EAST }` -> 0, 1, 2. The names are **global constants**; the optional *name* is only for grouping/readability. Explicit values must be numeric literals.

1.15.24 SWAP varID, varID

Exchanges the contents of two variables with no temporary. Bare operands (they are destinations). Each side may also be an **indirect** destination: `SWAP [a], [b]`, `SWAP a, [b]`.

1.15.25 FOR varID = varexpression TO varexpression [STEP expression] ... NEXT [varID]

Counted loop: runs *varID* from the start value to the end value. `STEP` is optional (defaults to `1`) and must be a **constant** (it may be negative). The counter is a byte and is written bare; read it in the body with `@varID`. If the start has already passed the limit, it runs zero times. See the flow control section for details.

1.15.26 SET varID TO varexpression

Assigns the value of *varexpression* to the variable *varID*.

1.15.27 SET varID TO WORD expression / DWORD expression / "string"

Assigns a **wide constant** to *varID* and the following consecutive variables: `WORD` writes 2 bytes (little-endian), `DWORD` 4 bytes and a string its character codes. See Wide constants. (Also valid with `LET varID = ...`.)

1.15.28 SET [varID] TO varexpression

Assigns the value of *varexpression* to the variable whose index corresponds to the contents of *varID*.

1.15.29 SET varID TO {varexpression1, varexpression2,...}

Assigns the value of *varexpression1* to the variable *varID*, *varexpression2* to the variable *varID*+1, and so on.

1.15.30 SET [varID] TO {varexpression1, varexpression2,...}

Assigns the value of *varexpression1* to the variable whose index corresponds to the content of *varID*, *varexpression2* to the variable whose index corresponds to the content of *varID*+1, and so on.

1.15.31 LET varID = varexpression

Assigns the value of *varexpression* to the variable *varID*.

1.15.32 LET [varID] = varexpression

Assigns the value of *varexpression* to the variable whose index corresponds to the content of *varID*.

1.15.33 LET varID = {varexpression1, varexpression2,...}

Assigns the value of *varexpression1* to the variable *varID*, *varexpression2* to the variable *varID*+1, and so on.

1.15.34 LET [varID] = {varexpression1, varexpression2,...}

Assigns the value of *varexpression1* to the variable whose index corresponds to the contents of *varID*, *varexpression2* to the variable whose index corresponds to the contents of *varID*+1, and so on.

1.15.35 LET varID += varexpression

Increments the variable *varID* by the value of *varexpression*. Equivalent to `LET varID = @varID + varexpression`.

1.15.36 LET [varID] += varexpression

Increments the variable whose index corresponds to the contents of *varID* by the value of *varexpression*.

1.15.37 LET arrayID(varexpression) += varexpression

Increments the element at position *varexpression* of the array *arrayID* by the value of the second *varexpression*.

1.15.38 LET varID -= varexpression

Decrements the variable *varID* by the value of *varexpression*. Equivalent to `LET varID = @varID - varexpression`.

1.15.39 LET [varID] -= varexpression

Decrements the variable whose index corresponds to the contents of *varID* by the value of *varexpression*.

1.15.40 LET arrayID(varexpression) -= varexpression

Decrements the element at position *varexpression* of the array *arrayID* by the value of the second *varexpression*.

1.15.41 END

Ends the adventure and restarts the computer.

1.15.42 CLEAR

Clears the defined text area.

1.15.43 CENTER

Sets the print cursor to the center of the line.

1.15.44 WAITKEY

Waits for the OK key to continue, displaying an animated wait icon. Ideal for separating paragraphs or screens.

1.15.45 OPTION GOTO ID

Creates a choice point that the user can select (see `CHOOSE`). If the choice is confirmed, it jumps to the *ID* label. If the screen is cleared, the choice point is deleted, and a maximum of 32 are allowed on a screen. Choice points are accumulated in the order they are declared in a list that is traversed according to the keystrokes pressed, and in the same order in which the choice points were declared. If it is the selected option in the menu, the value returned by `OPTIONVAL()` is its position within the list of menu options.

1.15.46 OPTION GOSUB ID

Creates a choice point that the user can select (see `CHOOSE`). If this option is confirmed, it jumps to label *ID*, returning after the `CHOOSE` when it encounters a `RETURN`. If the screen is cleared, the choice point is deleted, and a maximum of 32 are allowed on a screen. Choice points are accumulated in the order they are declared in a list that is traversed according to the keystrokes used, and in the same order in which the choice points were declared. If it is the selected option in the menu, the value returned by `OPTIONVAL()` is its position within the list of menu options.

1.15.47 OPTION VALUE(varexpression) GOTO ID

Works the same as `OPTION GOTO ID`, but we assign it a specific value that is returned by `OPTIONVAL()` if it is the chosen option in the menu.

1.15.48 OPTION VALUE(varexpression) GOSUB ID

Works the same as `OPTION GOSUB ID`, but we assign it a specific value that will be returned by `OPTIONVAL()` if it is the chosen option in the menu.

1.15.49 CHOOSE

Stops execution and allows the player to select one of the options currently on the screen. It will jump to the label indicated in the corresponding option.

Selection is made with the **O** and **P** keys for horizontal scrolling and **Q** and **A** for vertical scrolling. The corresponding directions on the Kempston joystick and the cursor keys can also be used.

When these keys are pressed, a pointer moves over the list of options, incrementing or decrementing them according to the current configuration (see MENUCONFIG). By default, only vertical scrolling is configured with the **Q** and **A** keys or **up** and **down** on the Kempston joystick or cursor keys. It is the user's responsibility to place the option points on the screen in a manner consistent with the configured menu movement.

Remember that if you clear the screen before this command, you will lose the options and a system error will be given.

1.15.50 CHOOSE IF WAIT expression THEN GOTO ID

It works exactly the same as `CHOOSE`, but with the exception that a timeout is declared, which if it expires without selecting any option, it jumps to the *ID* label. The timeout has a maximum of 65535 (16 bits).

1.15.51 CHOOSE IF CHANGED THEN GOSUB ID

It works exactly the same as `CHOOSE`, but a subroutine jump is made to the *ID* label each time the option is changed. Therefore, it is recommended that a `RETURN` is made as soon as possible to avoid slowness. To know the chosen option and the total number of options in our menu, we can use the `OPTIONSEL()` and `NUMOPTIONS` functions respectively.

1.15.52 OPTIONSEL()

Function that returns the currently selected option in the currently active menu. If we do not have an active menu, the result of this function is undefined.

1.15.53 NUMOPTIONS()

Function that returns the number of options in the currently active menu. If we do not have an active menu, the result of this function is undefined.

1.15.54 OPTIONVAL()

Function that returns the value assigned to the option selected and accepted in the menu, it is only updated when an option is chosen and the menu is exited.

1.15.55 CLEAROPTIONS

Deletes the options stored in the menu.

1.15.56 MENUCONFIG varexpression, varexpression, varexpression, varexpression

Configures the options menu. The parameters that can be configured are the following:

- The first parameter determines the increment or decrement of the selected option number when we press **P** and **O** respectively (or the left and right cursor keys or the joystick).
- The second parameter determines the increment or decrement of the selected option number when we press **A** and **Q** respectively (or the up and down cursor keys or the joystick).
- The third parameter is the option that will be selected at the beginning when the menu is started with `CHOOSE`. The default value is zero (the first option registered).
- The fourth parameter, if zero, does not show the selection icon and if non-zero, does.

The behavior when starting the interpreter is as if `MENUCONFIG 0,1,0,1` had been executed. **If you change the configuration for a menu, it is recommended to do so first, before setting the options.**

1.15.57 MENUCONFIG varexpression, varexpression, varexpression

If the fourth parameter is omitted, `MENUCONFIG x,y,d` is equivalent to `MENUCONFIG x,y,d,1`.

1.15.58 MENUCONFIG varexpression, varexpression

If the third and fourth parameters are omitted, `MENUCONFIG x,y` is equivalent to `MENUCONFIG x,y,0,1`.

1.15.59 CHAR varexpression

Prints the character indicated by its corresponding number.

1.15.60 REPCHAR expression, expression

Prints the character indicated in the first parameter as many times as the number indicated in the second parameter. Both values have a size of 1 byte, that is, val from 0 to 255. Also, if the number of times is zero, the character will be repeated 256 times instead of none.

1.15.61 TAB expression

Prints as many spaces as indicated in the parameter. It is the equivalent of `REPCHAR 32, expression`, but it takes up less memory.

1.15.62 PRINT varexpression

Prints the indicated numeric value.

1.15.63 PAGEPAUSE expression

Controls whether the player is prompted to continue after the current text area is completely filled, displaying an animated wait icon (parameter <> 0) or scrolling up the text and continuing printing (parameter = 0).

1.15.64 INK varexpression

Defines the character color (ink). Values 0-7, corresponding to the Spectrum colors.

1.15.65 PAPER varexpression

Defines the background color (paper). Values 0-7, corresponding to the Spectrum colors.

1.15.66 BORDER varexpression

Defines the border color, values 0-7.

1.15.67 BRIGHT varexpression

Turns brightness on or off (0 off, 1 on).

1.15.68 FLASH varexpression

Turns flashing on or off (0 off, 1 on).

1.15.69 NEWLINE expression

Prints newlines equal to the number specified in the parameter (0 is 256)

1.15.70 NEWLINE

Equivalent to `NEWLINE 1`.

1.15.71 BACKSPACE expression

Moves the print cursor back by the number of positions specified in the parameter (0 is 256), deleting the character at the new position and replacing it with a space. The character size used is the space (number 32). Note that if characters of different sizes than the space are used, this operation will not work properly.

1.15.72 BACKSPACE

Equivalent to `BACKSPACE 1`.

1.15.73 SFX varexpression

If a sound effects file has been loaded, plays the specified effect. If no such file has been loaded, the command is ignored.

1.15.74 PICTURE varexpression

Loads the image specified as the parameter into the buffer. For example, if 3 is specified, the file `003.CSC` will be loaded. The image is not displayed, which allows control over when the file is loaded.

1.15.75 DISPLAY varexpression

Displays the current content of the buffer on the screen. The parameter indicates whether or not the image is displayed; a value of 0 does not display it, and a value other than zero indicates yes. As many lines as have been defined in the corresponding image are displayed and the screen content will be overwritten.

1.15.76 BLIT varexpression, varexpression, varexpression, varexpression AT varexpression, varexpression

Copies a part of the image loaded in the buffer to the screen. We define with the parameters a rectangle within the buffer that will be copied to the screen from the indicated position.

The parameters, in order, are:

- Source column of the rectangle to be copied from the buffer.
- Source row of the rectangle to be copied from the buffer.
- Width of the rectangle to be copied from the buffer.
- Height of the rectangle to be copied from the buffer.
- Destination column on the screen.
- Destination row on the screen.

The unit of measurement used in all parameters is the 8x8 character and only the last two accept variables.

1.15.77 WAIT expression

Pauses. The parameter is the number of "frames" (50 per second) to wait, and is a 16-bit number.

1.15.78 PAUSE expression

Same as `WAIT`, but with the exception that the player can abort the pause by pressing the confirmation key.

1.15.79 TYPERATE expression

Indicates the pause that must occur between the printing of each character. Minimum 0, maximum 65535.

1.15.80 MARGINS expression, expression, expression, expression

Defines the screen area where the text will be written. The parameters, in order, are:

- Initial column.
- Initial row.
- Width (in characters).
- Height (in characters).

Sizes and positions are always defined as if they were 8x8 characters.

1.15.81 FADEOUT expression, expression, expression, expression

Fades out the screen area defined by the parameters, which in order are:

- Initial column.
- Initial row.
- Width (in characters).
- Height (in characters).

Sizes and positions are always defined as if they were 8x8 characters.

1.15.82 AT varexpression, varexpression

Places the cursor at a given position, relative to the area defined by the `MARGINS` command. The parameters, in order, are:

- Column relative to the origin of the text area.
- Row relative to the origin of the text area.

Positions are assumed to be 8x8 character size.

1.15.83 FILLATTR varexpression, varexpression, varexpression, varexpression, varexpression

We fill a rectangle on the screen with a given attribute value. The pixels are not altered. The parameters, in order, are:

- Column of origin of the rectangle to be filled.
- Row of origin of the rectangle to be filled.
- Width of the rectangle to be filled.
- Height of the rectangle to be filled.
- Value of attributes in Spectrum screen format, that is, a byte with bits in FBPPPIII format (F = Flash, B = Brightness, P = Paper, I = Ink).

1.15.84 PUTATTR varexpression, varexpression AT varexpression, varexpression

We put the attributes of an 8x8 character on the screen with given values. The pixels are not altered. The parameters, in order, are:

- Value of attributes in Spectrum screen format, that is, a byte with bits in FBPPPIII format (F = Flash, B = Brightness, P = Paper, I = Ink).
- Mask to be applied to the attributes already existing on the screen. If the bit is zero, we keep the one on the screen, and if it is one we use the new value.
- Column of the 8x8 character.
- Row of the 8x8 character.

1.15.85 PUTATTR varexpression AT varexpression, varexpression

Equivalent to `PUTATTR varexpression, 0xFF AT varexpression, varexpression`

1.15.86 GETATTR (varexpression, varexpression)

Function that returns the attribute value of an 8x8 character on the screen, in Spectrum screen format, that is, a byte with bits in FBPPPIII format (F = Flash, B = Bright, P = Paper, I = Ink).

The parameters, in order, are:

- Column of the 8x8 character.
- Row of the 8x8 character.

1.15.87 ATTRVAL (expression COMMA expression COMMA expression COMMA expression)

Function that returns the attribute value in Spectrum screen format, that is, a byte with bits in FBPPPIII format (F = Flash, B = Bright, P = Paper, I = Ink).

1.15.88 ATTRMASK (expression COMMA expression COMMA expression COMMA expression)

Function that returns a mask value, used in the `PUTATTR` and `FILLATTR` commands.

The parameters, in order, are:

- Ink mask. If the value is zero, we preserve the screen value, and if it is one we use the new value.
- Paper mask. If the value is zero, we preserve the screen value, and if it is one we use the new value.
- Bright mask. If the value is zero, we preserve the screen value, and if it is one we use the new value.
- Flash mask. If the value is zero, we preserve the screen value, and if it is one we use the new value.

1.15.89 RANDOM(expression)

Function that returns a random number between 0 and the value specified in **expression**. The maximum value is 255.

1.15.90 RANDOM()

Function that returns a random number between 0 and 255. It is the equivalent of `RANDOM(255)`.

1.15.91 RANDOM(expression, expression)

Function that returns a random number between the value indicated in the first parameter and the second, both inclusive.

1.15.92 INKEY(expression)

Function that returns the code of the key pressed. If the parameter is zero, it waits until a key is pressed. If it is non-zero, it returns the code of the key pressed at that moment, and zero if no key is pressed. **Only responds to keyboard presses, not the joystick**.

1.15.93 INKEY()

Function that waits until a key is pressed and returns the code of the key pressed. It is equivalent to `INKEY(0)`

1.15.94 KEMPSTON()

Function that returns the keys pressed on a connected Kempston joystick. The following bits will be set to one if the corresponding button is pressed:

- Bit 0 = Right.
- Bit 1 = Left.
- Bit 2 = Down.
- Bit 3 = Up.
- Bit 4 = Fire.

1.15.95 MIN(varexpression,varexpression)

Function that limits the value of the first parameter to the minimum indicated by the second. That is, if parameter 1 is less than parameter 2, parameter 2 is returned, otherwise 1 is returned.

1.15.96 MAX(varexpression,varexpression)

Function that limits the value of the first parameter to the maximum indicated by the second. That is, if parameter 1 is greater than parameter 2, parameter 2 is returned, otherwise 1 is returned.

1.15.97 YPOS()

Function that returns the current row in which the cursor is located in 8x8 coordinates.

1.15.98 XPOS()

Function that returns the current column where the cursor is located in 8x8 coordinates (Due to the nature of the variable width font, the value returned will be the 8x8 column where the cursor is currently located.)

1.15.99 WINDOW expression

Switches to the "window" indicated by the parameter and supports up to 8 windows (from 0 to 7). By default, the initial window is 0. CYD "windows" are a screen area defined by their original position, width, height, cursor position, and current attributes. This allows you to have different independent text areas on screen simultaneously without having to keep changing margins and cursor position to move from one area to another. Note that if windows overlap, the content of a window is not preserved if it is overwritten by another window.

1.15.100 CHARSET expression

Switches the character set used when printing text. With the parameter set to zero, the lower set (characters 0 through 127) is used, and with a non-zero value, the upper set (characters 128 through 255) is used. Remember that characters 0 through 32 and 127 through 143 are non-printable. This instruction does not affect REPCHAR and CHAR.

1.15.101 RANDOMIZE expression

Initializes the random number generator with the parameter value as the seed. Random number generation is not truly "random," and this can cause the generator to always return the same results if used in an emulator. Therefore, some source of randomness or entropy is needed. If we use this command with a value of zero as the parameter, the generator is initialized using the elapsed number of frames. This ensures randomness if it is executed in response to an arbitrary event, such as a key press. If we use a non-zero value, **RANDOM()** will always produce the same sequence of results, but it can be used as a procedural generator.

1.15.102 RANDOMIZE

This is equivalent to `RANDOMIZE 0`, which loads the random number generator with the elapsed number of frames.

1.15.103 TRACK varexpression

Loads the Vortex Tracker file into memory as a parameter. For example, if 3 is specified, it will load the music track from file `003.PT3`. If a track was previously loaded, it will be overwritten.

1.15.104 PLAY varexpression

If the parameter is non-zero and music is off, it plays the loaded music track. If it is currently playing, and 0 is passed as a parameter, it stops playing.

1.15.105 LOOP varexpression

Sets whether or not to repeat the music track after the end of the currently loaded music track. A value of 0 means false and non-zero means true.

1.15.106 RAMSAVE varID, expression

Copies from the variable specified in the first parameter, as many variables as were specified in the second parameter (if zero, it is considered as 256) to temporary storage. Before copying, the temporary storage is filled with zeros, so that the variables not copied will have this value in the temporary storage.

1.15.107 RAMSAVE varID

This is the equivalent of `RAMSAVE varId, 0`, that is, it copies all the variables from the first parameter to the end to the temporary storage.

1.15.108 RAMSAVE

This is the equivalent of `RAMSAVE 0, 0`, that is, it copies all the variables to the temporary storage.

1.15.109 RAMLOAD varID, expression

This is the inverse operation of `RAMSAVE`. It copies from temporary storage to the variables, from the variable indicated in the first parameter and as many variables as those indicated in the second (if it is zero, it is considered as 256).

1.15.110 RAMLOAD varID

This is the equivalent of `RAMLOAD varId, 0`, that is, it copies all the variables from the first parameter to the end from the temporary storage.

1.15.111 RAMLOAD

This is the equivalent of `RAMLOAD 0, 0`, that is, it copies all variables from the storage

1.15.112 SAVE varexpression, varId, expression

Saves a series of variables to tape or disk. The first parameter indicates the file number to save, the second parameter is the initial variable of the range of variables to save the file, and the third parameter is the number of variables to save in that range (if zero, it is considered 256). Anything in the temporary storage that was loaded with `RAMSAVE` will be removed, since it will be used to store the file temporarily.

- In the case of disk, the file with the indicated three-digit number and `.SAV` extension will be saved. For example, the file to load with 16 would be `016.SAV`. If the file already exists, it will be overwritten.
- In the case of tape, it will be saved to tape. The system will immediately start saving, so the author is responsible for telling the user that he is going to record and that he has to put the cassette unit (or equivalent) to record. The process can be interrupted with the **BREAK** key.

The result of the operation can be checked with the `SAVERESULT()` function.

1.15.113 SAVE varexpression, varId

This is the equivalent of `SAVE varexpression, varId, 0`, that is, it saves all variables from the second parameter to the end.

1.15.114 SAVE varexpression

This is the equivalent of `SAVE varexpression, 0, 0`, that is, it saves all variables

1.15.115 LOAD varexpression

Loads a series of variables from tape or disk. The parameter indicates the file number to load, and the range of variables that was indicated when saving the file will be loaded. Anything in the temporary storage that was loaded with `RAMSAVE` will be deleted, since it will be used to store the file temporarily.

- In the case of disk, the file with the indicated three-digit number and `.SAV` extension will be loaded. For example, the file to be loaded with 16 would be `016.SAV`.
- In the case of a tape, the first block without a header that is found will be loaded, so the user must advance the tape to the correct position. The system will immediately start loading, so the author is responsible for indicating to the user that it is going to load and that he has to put the cassette unit (or equivalent) to play. The process can be interrupted with the **BREAK** key.

The result of the load can be consulted with the `SAVERESULT()` function.

1.15.116 SAVERESULT()

Function that returns the result of the last `SAVE` or `LOAD` operation. The possible values are the following:

0. Correct result.
1. Error when loading/saving on tape/disk. In the case of a tape, operation interrupted with **BREAK**.
2. The saved game does not correspond to the current game.
3. The selected slot of the saved game does not correspond to the one requested.
4. Error checking the saved game file.

Any other value results in undefined results.

1.15.117 ISDISK()

Function that returns 1 if the interpreter is the Plus3 version and 0 otherwise.

1.15.118 LASTPOS(ID)

Function that returns the last accessible position of the given array. For all purposes, the size of the array minus 1.

1.16 Images

The engine supports a maximum of 256 images, plus whatever fits on disk or memory. They must be named with a 3-digit number, corresponding to the image number that will be invoked from the program. For example, image 0 should be called `000.scr`, image 1 `001.scr`, and so on up to 255. Images must be in the Spectrum display format.

To display images, the compiler compresses the SCR format files from the Spectrum display into compressed files with a `csc` extension. The compression process can be lengthy, so the compiler detects whether for each `scr` file, there is a more recent equivalent file with a `csc` extension in the images directory. If so, it skips this file, and if not, it recompresses it, generating the `csc` file again.

You can configure the number of horizontal lines to display in the image using the corresponding parameter in the compiler to further reduce the size. Additionally, if it detects that the right half of the image is mirrored by the left half, it discards the left half to further reduce the size, although we can force this behavior with any image. To do this, create a file called `images.json` in the images directory.

The structure of the `images.json` file is as follows:

```
[
{
"id": 0,
"num_lines": 192,
"force_mirror": false
},
{
"id": 1,
"num_lines": 64,
"force_mirror": true
```

```
}
]
```

It is a list of records with the following fields:

- `id` : This is the image number corresponding to this entry. In the example, `"id":0` corresponds to the image `000.scr` and `"id":1` corresponds to the image `001.scr`.
- `num_lines`: Indicates the number of lines to include in the compressed file. The minimum is 1 and the maximum (full screen) is 192.
- `force_mirror`: With a value of `true`, we force the image to be considered symmetrical. This will cause the right half of the image to be discarded, and when it is decompressed in memory, the mirrored left half will be drawn in its place. Otherwise, this field must be set to `false`.

We must specify in the JSON as many records as images we want to define the behavior. Images not defined in the `images.json` file will be treated with normal behavior, which is to use the number of lines defined with the `-il` parameter, or 192 by default, and to use mirroring only for images detected as symmetrical.

There are two commands required to display an image: the `PICTURE n` command will load image number n into a buffer. That is, if you type `PICTURE 1`, it will load the `001.CSC` file into the buffer. Any operation on an image requires first loading it into the buffer. This is useful for controlling when the image should be loaded, as it will require waiting for the image to load from disk, in addition to allowing some time to decompress the image on both tape and disk. Therefore, it is recommended to run this command at an appropriate time, for example, at the start of a chapter. If we try to load an image that doesn't exist, we'll receive an error 1, and if we try to load an image whose file doesn't exist on disk, we'll generate a disk error 23.

To display an image loaded into the buffer, we use `DISPLAY n`, where n must be zero to execute. The image displayed will be the last one loaded into the buffer (if it exists). The image starts painting from the top left corner of the screen, drawing the same number of lines as the file compression commands, overwriting everything previously on the screen.

There is also an alternative method for displaying images: the `BLIT` command. This allows us to copy a fragment of the image onto the screen instead of the entire image, and also allows us to specify its position on the screen. The command format is `BLIT xo, yo, width, height AT xd, yd`, and you can find more details about it in its corresponding section. `BLIT` is more versatile than `DISPLAY`, but it is also slower, and it also requires an image to be loaded into the image buffer with `PICTURE` beforehand, and it will overwrite anything that was on the screen at the drawing position.

1.17 Sound effects

Adding sound effects with the beeper is very simple. To do this we have to create an effects bank with the BeepFx utility. We must export the effects file as an assembler file using the `File->Compile` option from the top menu. In the floating window that appears, we must make sure that we have `Assembly` and `Include Player Code` selected, the rest of the options are irrelevant, since the compiler will modify the source file to embed it in the interpreter.

To invoke an effect, we use the command `SFX n`, where n is the number of the effect to be played. If

this command is called without an effect file loaded, it will be ignored and execution will continue.

It is not contemplated to call an effect number that does not exist in the included file.

1.18 Vortex Tracker Music

The `CYD` engine allows you to play music modules created with Vortex Tracker in `PT3` format. Its operation replicates the image loading mechanism; that is, modules must be named with three digits representing the track number that the player will load with the `TRACK` command. For example, if the player encounters the `TRACK 3` command, they will search for the track in the `003.PT3` file and load the module into memory for playback. Similarly, the maximum number of modules that can be loaded is 256 (from 0 to 255), and the module cannot exceed 16 kilobytes for tape and 8 kilobytes for Plus3.

Once a module is loaded, it can be played with the `PLAY` command. As indicated in the command reference, if the parameter is non-zero, the module will be played; if it is equal to zero, playback will stop.

When the end of the module is reached, playback will automatically stop, but we can change this behavior with the `LOOP` command. Again, as indicated in the reference, if the parameter is equal to zero, playback will stop when the end is reached, as indicated above. However, if the parameter is non-zero, the module will restart from the beginning.

1.19 WyzTracker Music

`CYD` also allows you to play music modules created with WyzTracker v2.0. It works similarly to Vortex Tracker Melodies and images; module file names must be three-digit numbers from 0 to 255, so with `TRACK 3`, we would load the corresponding `003.mus` module.

Once a module is loaded, it can be played with the `PLAY` command. As indicated in the command reference, if the parameter is non-zero, the module will play; and if it is equal to zero, playback will stop.

Unlike with Vortex Tracker modules, the `LOOP` command is not fully supported, as the modules themselves define whether they will loop or not.

Due to the peculiarities of the WyzTracker format, several considerations must be taken into account that will affect the use of the engine with this music player:

- All modules must share the same instruments. The instrument file must be named `instruments.asm` and must be stored with the module files in the `TRACKS` directory.
- *The entire bank 1 of the Spectrum's memory will be reserved, where the player code, instruments, and melodies will be stored. This means that 16 KB less will be available for the tape version, and 8 KB less for the Plus3 version.
- The different melodies will be compressed to save space, and each time one is loaded, it will be decompressed into the remaining space in bank 1. The compiler will generate an error if any of the included melodies do not fit in this remaining space.
- Sound effects are not supported.

1.20 How to generate an adventure

The first thing to do is to generate a script for the adventure using any text editor using the syntax described above. It is HIGHLY recommended to write the script before programming the logic, since it is important to have the text outlined beforehand to ensure proper compression (more details later).

It is important that the file is encoded in UTF-8, but it must be taken into account that characters over 128 will not be printed correctly and only the characters specific to Spanish are allowed, indicated in the Character Set section, which will be converted to the codes indicated there.

Once we have the adventure, we use the `CYDC` compiler to generate a DSK or TAP file. The compiler looks for the best abbreviations to compress the text as much as possible. The process can be very long depending on the size of the adventure. That is why it is important to have the adventure outlined beforehand, to carry out this process at the beginning. We will compile the file with the parameter `-T`, so that with `-T abbreviations.json`, for example, we will export the abbreviations found to the file *abreviations.json*.

From this point on, if we run the compiler with the parameter `-t abbreviations.json`, it will not search for abbreviations and will use the ones we had already found before, so the compilation will be almost instantaneous. When we consider that the adventure is finished, we can perform a new search for abbreviations to try to achieve a little more compression.

To add sound effects, images and melodies, see the corresponding sections.

The process is quite simple, but it has some dependent steps, so it is recommended to use BAT files (Windows) or shell scripts (Linux, Unix) or the Make utility (or similar) to speed up development.

To make things easier, a program called `make_adventure.py` is included that will do all of these tasks for us, as well as the corresponding `make_adv.cmd` scripts for Windows and `make_adv.sh` for UNIX to run it.

The `make_adventure.py` program will automatically search for and compress SCR files that conform to the established filename format (a 3-digit number from 0 to 255) in the `.\IMAGES` directory. It will do the same with any modules in the `.\TRACKS` directory that conform to the filename format. It will then compile the `test.txt` file and generate the `tokens.json` file with the abbreviations, if it doesn't already exist. If you want to regenerate the abbreviations file (it's recommended to do so when you're running out of memory or are finishing the adventure), simply deleting it will tell the compiler to regenerate it. It will also automatically search for a sound effects file called `SFX.ASM` that should be generated with **BeepFX**, and if a JSON file with the character set called `charset.json` exists, it will use that as well.

It also supports language selection for output messages. You can specify a language with the `--lang {en,es}` parameter or by setting the `CYD_LANG` environment variable.

This program needs the `dist` and `tools` directories with their contents to perform the process. The peculiarities of each operating system are detailed below:

1.20.1 make_adventure.py (CLI helper)

`make_adventure.py` is a helper wrapper around `cydc_cli.py` that standardizes project paths and automates token handling.

```
make_adventure.py [options] {48k,128k,plus3,mld,mld128} [SJASMPLUS_PATH]
```

Main options: - `--lang {en,es}`: Output language for helper messages. - `-n, --name NAME`: Adventure base name (expects `NAME.cyd`, default `test`). - `-o, --output-path OUTPUT_PATH`: Directory for output files. - `-img, --images-path`, `-trk, --tracks-path`, `-sfx, --sfx-asm-file`, `-scr, --load-scr-file`. - `--tok, --tokens-file`: Token file path. If it does not exist, `-T` is used automatically; if it exists, `-t` is used. - `-chr, --charset-file`: Character set JSON path (used if found). - `-il, --image-lines`, `-l`, `-L`, `-s`, `-S`, `-trim`, `-code`, `--no-strict-colons`, `-pause`, `-wyz`, `-720`.

Note: after successful `plus3` builds, temporary files `SCRIPT.DAT`, `DISK`, and `CYD.BIN` are cleaned automatically.

1.20.2 Windows version

As an example, the file `make_adv.cmd` has been included in the root of the repository, which will compile the sample adventure included in the file `test.cyd`.

You can use it as a base to easily create your own adventure. You can customize the behavior by modifying some variables in the script header:

```
REM ---- Configuration variables ----------

REM Name of the game
SET GAME=test
REM This name will be used as:
REM   - The file to compile will be test.cyd with this example
REM   - The name of the TAP file or +3 disk image

REM Target for the compiler (48k, 128k for TAP, plus3 for DSK, mld or mld128 for MLD)
SET TARGET=48k

REM Number of lines used on SCR files at compressing
SET IMGLINES=192

REM Loading screen
SET LOAD_SCR="./IMAGES/LOAD.scr"

REM Parameters for compiler
SET CYDC_EXTRA_PARAMS=

REM Run emulator after compilation (none/internal/default)
REM     -None: Do nothing
REM     -internal: run the compiled file with Zesarux (must be on the directory .\tools\zesarux\)
REM     -default: run the compiled file with the program set on Windows to run its extension
SET RUN_EMULATOR=none
```

```
REM Backup CYD file after compilation (yes/no) on ./BACKUP directory.
SET BACKUP_CYD=no

REM Maximum number of backup files to keep (0 = unlimited)
SET BACKUP_MAX_FILES=0

REM -----------------------------------
```

- The variable `TARGET` is the output system and format, with these possible options: – 48k: Generates a TAP file for Spectrum 48K, without AY music support. – 128k: Generates a TAP file for Spectrum 128K. – plus3: Generates a DSK file for Spectrum +3, with higher capacity and dynamic loading of resources. – mld: Generates a strict 48K (non-banked) MLD file for Dandanator. – mld128: Generates an MLD file for Dandanator targeting Spectrum 128K. The RAM banks are used for music and for the `DIM` arrays (which must be writable); the rest of the adventure data (text/images/bytecode) is read directly from the cartridge slots.
- The variable `IMGLINES` is the number of horizontal lines of the image files to be compressed. By default it is 192 (the full Spectrum screen)
- The variable `LOAD_SCR` is the path to a SCR file (Spectrum screen) with the screen to be used during loading.
- The variable `CYDC_EXTRA_PARAMS` is used to add extra parameters in the call to the compiler cydc.
- The variable `RUN_EMULATOR` indicates if we want the compiled program to be executed under an emulator with the following possible values: – none: If we do not want it to do this. – internal: Executes the compiled file under ZEsarUX in `.\tools\ZEsarUX_win-11.0\`. – default: If the file extension is associated under Windows with another emulator, it will be executed with this one.
- The variable `BACKUP_CYD` with the value `yes` makes a backup copy of the current file inside the `.\BACKUP` directory. Each copy adds the date on which it was created to the file name.
- The variable `BACKUP_MAX_FILES` limits how many backups are kept per game (`0` means unlimited).

The script will produce a DSK or TAP file (depending on the format selected in `TARGET`) that you can run with your favorite emulator. But if you want to speed up the work even more, if you download Zesarux and install it in the `.\tools\ZEsarUX_win-11.0` folder, after compilation it will run automatically with the appropriate options.

1.20.3 GUI Compiler (make_adventure_gui v1.0.0)

For those who prefer a graphical interface instead of editing scripts or command lines, the **make_adventure_gui** tool provides a cross-platform GUI for compiling Choose Your Destiny adventures.

Features: - Cross-platform support (Windows, Linux, macOS with Python 3.11+) - Embedded Python on Windows (no separate Python installation needed) - 26 configurable options including compilation targets, paths, and post-build actions - Settings persistence (remembered between sessions) - Full internationalization support (English and Spanish) with runtime language switching via dropdown - Real-time compilation output display - Auto-hide console window on Windows for clean startup

Launching the GUI:

Windows:

```
make_adventure_gui.cmd
```

Linux/macOS:

```
./make_adventure_gui.sh
```

Language Selection:

The GUI supports Spanish and English languages with automatic detection from your system locale. You can change the language at any time using the **Language** dropdown in the header - all UI text will update immediately. The language preference is saved and restored automatically.

You can also pre-set the language with the `CYD_LANG` environment variable before launching:

```
REM Windows
set CYD_LANG=es
make_adventure_gui.cmd
```

```
# Linux/macOS
export CYD_LANG=es
./make_adventure_gui.sh
```

Configurable Options:

Category	Options
Project	Game name, Target (48k/128k/plus3/mld/mld128)
Paths	Output directory, Images path, Tracks path, SFX file, Loading screen, Tokens file, Character set, SjASMPlus executable
Compiler	Image display lines, Text abbreviation limits, Superset limit, Verbose mode, Slice texts between banks, Trim interpreter code, Show bytecode, Strict colon compatibility, Pause-after-load, WyzTracker support, 720KB disk
Post-Build	Run emulator after compilation, Backup CYD file
Appearance	Font size, Log font family/size, Log text/background colors

All compiled output files are placed in the specified output directory, and the GUI will display detailed compilation messages for debugging if needed.

1.20.4 Linux, BSDs and Unices version

As an example, the file `make_adv.sh` has been included in the root of the repository, which will compile the sample adventure included in the file `test.cyd`.

You can use it as a base to easily create your own adventure. You can customize the behavior by modifying some variables in the script header:

```
# ---- Configuration variables ----------
# Name of the game
GAME="test"
# This name will be used as:
#   - The file to compile will be test.cyd with this example
#   - The name of the TAP file or +3 disk image
#
# Target for the compiler (48k, 128k for TAP, plus3 for DSK, mld or mld128 for MLD)
TARGET="48k"
#
# Number of lines used on SCR files at compressing
IMGLINES="192"
#
# Loading screen
LOAD_SCR="./LOAD.scr"
#
# Parameters for compiler
CYDC_EXTRA_PARAMS=

# Run emulator after successful compilation (none/internal/custom)
RUN_EMULATOR="none"

# Custom emulator command (used when RUN_EMULATOR=custom)
CUSTOM_EMULATOR_CMD="fuse \${OUTPUT_FILE}"

# Path to ZEsarUX (used when RUN_EMULATOR=internal)
ZESARUX_PATH="./tools/ZEsarUX_linux/zesarux"

# Backup CYD file after compilation (yes/no)
BACKUP_CYD="no"

# Maximum number of backup files to keep (0 = unlimited)
BACKUP_MAX_FILES=0
# ---------------------------------------
```

- The variable `TARGET` is the output system and format, with these possible options: – 48k: Generates a TAP file for Spectrum 48K, without AY music support. – 128k: Generates a TAP file for Spectrum 128K. – plus3: Generates a DSK file for Spectrum +3, with higher capacity and dynamic loading of resources. – mld: Generates a strict 48K (non-banked) MLD file for Dandanator. – mld128: Generates an MLD file for Dandanator targeting Spectrum 128K. The RAM banks are used for music and for the `DIM` arrays (which must be writable); the rest of the adventure data (text/images/bytecode) is read directly from the cartridge slots.
- The variable `IMGLINES` is the number of horizontal lines of the image files to be compressed. By default it is 192 (the full Spectrum screen)

- The variable `LOAD_SCR` is the path to a SCR file (Spectrum screen) with the screen to be used during loading.
- `RUN_EMULATOR` supports `none`, `internal` (ZEsarUX), or `custom`.
- `CUSTOM_EMULATOR_CMD` is used when `RUN_EMULATOR=custom`.
- `ZESARUX_PATH` defines the emulator executable path for `internal` mode.
- `BACKUP_CYD` and `BACKUP_MAX_FILES` control automatic backup generation and retention.

1.21 Examples

A comprehensive set of examples is available to test the engine's capabilities and learn from them. Each example is designed to illustrate specific features of the CYD language, from basic concepts to advanced techniques.

1.21.1 Basic Examples

1.21.1.1 `examples\test` - Engine Introduction **Level:** Beginner | **Requirements:** ZX Spectrum 128K

This is the perfect starting point. It demonstrates the fundamental concepts of the engine: - **Options and navigation system:** Implements a basic menu with `OPTION` and `CHOOSE`, showing how to create story branching. - **Variables and operations:** Uses variables to keep track of objects ("gamusinos") and perform basic arithmetic operations. - **Music and sound:** Includes a sample song in the `TRACKS` directory and demonstrates how to play music with `PLAY` and `LOOP`. - **Images:** Loads and displays SCR images from the `IMAGES` directory using `PICTURE` and `DISPLAY`. - **Flow control:** Illustrates the use of `GOTO`, `LABEL`, `WAITKEY`, and screen clearing with `CLEAR`.

This example is an expanded version of the code shown in the Syntax section of the manual.

1.21.1.2 `examples\multicolumn_menu` - Multi-column Menus **Level:** Beginner

Demonstrates how to organize options in multiple columns to create more compact and visually appealing menus: - **Menu configuration:** Uses `MENUCONFIG 1,2` to define a menu with 1 row and 2 columns. - **Visual layout:** Shows how to place options side-by-side instead of vertically. - **Use cases:** Ideal for games with many options, character selection, inventories, etc.

1.21.1.3 `examples\guess_the_number` - Guessing Game **Level:** Beginner-Intermediate

A complete "guess the number" game that illustrates: - **Random numbers:** Generates a secret number using `RANDOM(1, 32)`. - **6-column menu:** Uses `MENUCONFIG 1,6` to create a compact menu with 32 options (numbers 1 to 32). - **Game logic:** Compares the player's guess with the secret number and provides hints ("higher"/"lower"). - **Variable management:** Demonstrates how to use variables to store the secret number and player attempts.

This example is excellent for learning how to create simple interactive games.

1.21.1.4 `examples\input_test` - Keyboard Input and Arrays **Level:** Intermediate

Technical example showcasing advanced data handling features: - **Keyboard reading:** Captures user input using `INKEY()` to read pressed keys. - **Array simulation:** Uses indirection with brackets `[@ptr]` to create and manipulate character arrays. - **String management:** Implements functions to print and edit text strings up to 16 characters. - **Subroutines:** Demonstrates the use of `GOSUB`

and `RETURN` to create reusable functions (`inputStr` and `printStr`). - **Interactive editing:** Allows character deletion with DELETE and displays a blinking cursor.

This example is fundamental for understanding how to work with complex data and create custom input interfaces.

1.21.1.5 `examples\math_library` - 16/32-bit arithmetic **Level:** Intermediate

Shows how to use the `lib/math16_32.cyd` library to work with numbers that exceed 8 (and 16) bits: - **Including libraries:** Adds the library with `INCLUDE` at the start of the program. - **Overflow-free multiplication:** Uses `mul1632` (`C(32) = A(16) * B(16)`) to compute a score. - **32-bit addition and printing:** Combines `add32` and `print32` to show the result in decimal. - **Loading wide literals:** Uses the multiple assignment `SET reg TO {b0, b1, ...}`.

1.21.1.6 `examples\strings_library` - Text strings **Level:** Intermediate

Shows how to use the `lib/strings.cyd` library for string input and handling: - **Keyboard input:** Reads a name with `strInput` (cursor, ENTER, DELETE). - **Printing and length:** Shows the string with `strPrint` and its length with `strLen`. - **Configurable buffer:** The author chooses where and how large the string is (`stBase`, `stLen`).

1.21.1.7 `examples\windows` - Multiple Windows **Level:** Beginner-Intermediate

Illustrates the window system for dividing the screen into different areas: - **Window definition:** Creates 4 windows using `WINDOW` and `MARGINS`. - **Independent areas:** Each window has its own text color (`INK`) and can be cleared independently. - **Window navigation:** Demonstrates how to switch between windows to print text in different areas. - **Use cases:** Useful for creating SCUMM-like interfaces, separating description from actions, displaying permanent stats, etc.

1.21.2 Intermediate Examples

1.21.2.1 `examples\ETPA_ejemplo` - "Choose Your Own Adventure" Book **Level:** Intermediate | **Requirements:** Images in `IMAGES`

Complete implementation of the first paragraphs of a "Choose Your Own Adventure" book (CYOA): - **Branching narrative:** Demonstrates how to create a story with multiple paths and endings using `OPTION` and `GOTO`. - **Configuration variables:** Uses variables to control display modes (text-only vs. with images, image position). - **Color system:** Differentiates between narrative text and options using different ink colors. - **Image management:** Loads SCR images to illustrate key scenes in the adventure. - **Modular structure:** Organizes code with labels for each paragraph/section (`LABEL s1`, `LABEL s2`, etc.). - **Screen subroutines:** Uses `GOSUB Pantalla` to manage different presentation modes.

This example is ideal for creating conversational adventures or interactive gamebooks.

1.21.2.2 `examples\include_demo` - Multi-file Organization **Level:** Intermediate

Demonstrates the use of the `INCLUDE` system to organize large projects: - **Code separation:** The main file `main.cyd` includes other files with `INCLUDE "file.cyd"`. - **Modular organization:** - `variables.cyd`: Variable declarations - `common.cyd`: Shared subroutines - `intro.cyd`: Introduction sequence - `chapter1.cyd` and `chapter2.cyd`: Independent chapters - **Maintainability:** Facilitates

teamwork and complex project organization. - **Reusability:** Common subroutines can be shared between multiple chapters.

See the `examples\include_demo\README.md` file for more details.

1.21.2.3 `examples\import_demo` - Native routines (IMPORT / CALL) **Level:** Advanced

Shows how to call a Z80 routine from a script with `IMPORT` / `CALL`: - **Native code:** a tiny `peek.asm` reads a byte from an arbitrary memory address, something the virtual machine cannot do on its own. - **ABI:** the routine is entered with `DE = FLAGS` and exchanges data through variables. - **Advanced, at your own risk:** see the "Native routines (IMPORT / CALL)" section above.

See the `examples\import_demo\README.md` file for more details.

1.21.2.4 `examples\inline_asm` - Inline assembler (ASM / EXPORTS / USES / CYD_CALL) **Level:** Advanced

Write native Z80 routines **inside the script itself** with `ASM … ENDASM` and combine them into a small "library":

- **`ASM` / `EXPORTS`:** one block with two entry points that share code, no separate file.
- **Array broker:** the routines read a script `DIM` array by name (`ARR_stats`) with `CYD_ARR_MAP`, working even when the array is in a paged bank.
- **`USES` / `CYD_CALL`:** one routine calls others in another block (and another bank) through the resident trampoline.
- **Dead code:** routines nothing reaches are dropped automatically.

See the `examples\inline_asm\README.md` file for more details.

1.21.3 Advanced Graphics Examples

1.21.3.1 `examples\blit` - Introduction to BLIT **Level:** Intermediate | **Requirements:** Images in `IMAGES`

Introduction to the `BLIT` command for copying sprites and graphics: - **Basic graphics copying:** Uses `BLIT x, y, width, height AT col, row` to copy image sections. - **Rendering loop:** Combines `BLIT` with `WHILE` loops to fill the screen with a pattern. - **Fundamental concept:** Foundation for understanding animations and more complex graphics.

1.21.3.2 `examples\blit_island` - Advanced Graphics **Level:** Intermediate-Advanced

More sophisticated example of `BLIT` usage: - **Scene composition:** Builds complex scenes by combining multiple sprites. - **Optimization:** Demonstrates techniques for efficient drawing. - **Multiple sprites:** Management of several graphic elements on screen.

1.21.3.3 `examples\Rocky_Horror_Show` - Character Animation **Level:** Advanced

Implements an animated intro screen: - **Frame-by-frame animation:** Uses `BLIT` to create animations by changing frames. - **Precise timing:** Combines `WAIT` with `BLIT` to control animation speed. - **Large sprites:** Handling larger characters on screen.

1.21.3.4 `examples\CYD_presents` - Complex Visual Effects **Level:** Advanced

Example for visual effects: - **Presentation effects:** Implements Spectrum "demo scene" style animations. - **Synchronization:** Coordinates multiple visual elements with music. - **Advanced techniques:** Combines `BLIT`, `WAIT`, `PICTURE`, and color effects.

The series `blit` → `blit_island` → `Rocky_Horror_Show` → `CYD_presents` forms a natural progression for mastering graphics in CYD.

1.21.3.5 `examples\Golden_Axe_select_character` - Dynamic Selection with Colors **Level:** Advanced | **Requirements:** Images in `IMAGES`

Recreates a Golden Axe-style character selection screen: - **Option change detection:** Uses `CHOOSE IF CHANGED THEN GOSUB` to execute code when the player moves the cursor. - **Dynamic color effects:** Uses `FILLATTR` to change color attributes of screen areas in real-time. - **Selection highlighting:** Illuminates the selected character with a blinking effect using loops. - **Fade out effects:** Implements a fade-out effect by dividing the screen into 4 sections with `FADEOUT`. - **Contextual information:** Displays descriptive text that changes based on the selected character. - **Helper functions:** `OPTIONSEL()` returns the index of the currently selected option.

This example is perfect for creating visually appealing and interactive selection menus.

1.21.4 Complete Project Examples

1.21.4.1 `examples\SCUMM_16` - SCUMM/LucasArts-style Interface **Level:** Advanced

Complete implementation of a SCUMM-style graphic adventure interface (like Monkey Island or Maniac Mansion): - **Multiple windows:** Divides the screen into graphics area, text area, and verb area. - **Verb menu:** Implements a 9-verb menu (3x3) at the bottom of the screen. - **Two-level system:** First level for selecting verb, second level for selecting object/direction. - **Sprite display:** Uses `BLIT` to copy location image and verbs. - **Object management:** Prints a list of visible objects in the location. - **Custom charset:** Loads a 4x8 character set for more compact text using `CHARSET 1`. - **Visual feedback:** Changes the color of the selected verb with `FILLATTR`. - **Modular organization:** Uses subroutines to print objects, directions, and manage selection.

This example is an excellent template for creating point-and-click graphic adventures.

1.21.4.2 `examples\Delerict` - Complete Adventure with Custom Engine **Level:** Advanced-Expert | **Requirements:** Images in `IMAGES`

Skeleton of a complete space adventure with all necessary systems: - **Complete object system:** - Object properties (can be carried, worn, opened, turned on, etc.) - Bit mask for object flags using binary constants (`0b00000001`) - Object location management (carried, worn, on the ground, etc.) - **Location system:** - Location descriptions - Exits in 8 directions (N, S, E, W, Up, Down, Enter, Exit) - Location flags (doors open/closed, etc.) - **Command system:** - 10 implemented verbs (Examine, Use, Take, Drop, Wear, Take Off, Turn On, Turn Off, Open, Close) - Action parser with direct object - Action validation based on object properties - **Extensive use of constants:** Defines constants with `CONST` to make code more readable. - **Scalable architecture:** Designed to easily add new objects, locations, and commands. - **1428 lines of code:** Demonstrates how to structure large projects.

This is the most complex example and serves as a foundation for creating traditional text adventures or hybrid (text + graphics) adventures. Studying this code is a masterclass in structured programming with CYD.

1.21.5 How to Use the Examples

To test without compiling: Each directory includes a precompiled `.tap` file that you can load directly into your favorite ZX Spectrum emulator.

To compile them yourself: 1. Copy the contents of the example folder to the root directory of the CYD distribution. 2. **Important:** If you have your own files, save them first, as they will be overwritten. 3. Run `make_adv.cmd` (Windows) or `./make_adv.sh` (Linux/macOS). 4. The resulting `.tap` file will be ready to test.

Recommended study order: It's recommended to study the examples in this order for a gradual learning curve: 1. `test` → Basic concepts 2. `multicolumn_menu` → Menus 3. `guess_the_number` → Game logic 4. `windows` → Split interface 5. `input_test` → Advanced input 6. `ETPA_ejemplo` → Branching narrative 7. `include_demo` → Code organization 8. `blit` → `blit_island` → `Rocky_Horror_Show` → `CYD_presents` → Progressively complex graphics 9. `Golden_Axe_select_character` → Dynamic visual effects 10. `SCUMM_16` → LucasArts-style interface 11. `Delerict` → Template for an adventure engine

---

1.22 Character set

The engine supports a character set of 256 characters, with a height of 8 pixels and variable width size from 1 to 8 pixels. The default character set included is 6x8 in size, along with an alternative set of 4x8 size starting from character 144. This is the default character set, ordered from left to right and top to bottom:

The characters correspond to standard ASCII, except for the Spanish characters, which correspond to the following positions for both character sets:

Character	Position 6x8	Position 4x8
'a'	16	144
'¡'	17	145
'¿'	18	146
'«'	19	147
'»'	20	148
'á'	21	149
'é'	22	150
'í'	23	151
'ó'	24	152
'ú'	25	153
'ñ'	26	154
'Ñ'	27	155
'ç'	28	156
'Ç'	29	157
'ü'	30	158

Character	Position 6x8	Position 4x8
'Ü'	31	159

Characters above the value 127 (starting from zero) to 143 (both included) are special. They are used as icons in options, i.e. where an option appears when the `OPTION` command is processed, and as wait indicators with a `WAITKEY` or when changing pages if the `PAGEPAUSE` command is active.

- Character 127 is the character used when an option is not selected in a menu. (In red in the screenshot below)
- Characters 128 to 135 form the animation cycle of an option selected in a menu. (In green in the screenshot below)
- Characters 135 to 143 form the animation cycle of the wait indicator. (In blue in the screenshot below)

The compiler has two parameters, `-c` to import a new character set, and `-C` to export the currently used character set, in case it can be used as a template or for customizations.

This is the import/export format for the character set:

```
[{"Id": 0, "Character": [255, 128, ...], "Width":8}, {"Id": 1, "Character": [0, 1, ...], "Width":6}, ..
```

It is a JSON with an array of records with three fields:

- *Id*: character number from 0 to 255, which cannot be repeated.
- *Character*: an array of 8 numbers that corresponds to the value of the bytes of the character and, therefore, there cannot be values greater than 255. Each byte corresponds to the pixels of each line of the character.
- *Width*: an array with the width in pixels of the character (the values cannot be less than 1 or greater than 8). Since the size of each character line in the above field is 8, any extra pixels on the right will be discarded.

To make it easier to create an alternative character set, the CYD Character Set Converter or cyd_chr_conv tool has been included. This tool allows you to convert fonts in `.chr`, `.ch8`, `.ch6` and `.ch4` formats created with ZxPaintbrush into the aforementioned JSON format. In addition, in the `assest` directory of the distribution you can find the `default_charset.chr` file with the default font so you can edit and customize it with that program.

1.23 Error codes

The application may generate errors at runtime. There are two types of errors, disk errors and engine errors.

Engine errors are, as their name indicates, errors that occur when the engine detects an anomalous situation. They are the following:

- System Error 1: The accessed resource does not exist. This is because PICTURE or TRACK is being executed with an index that does not exist in the adventure, since the corresponding

Figure 2: Default character set

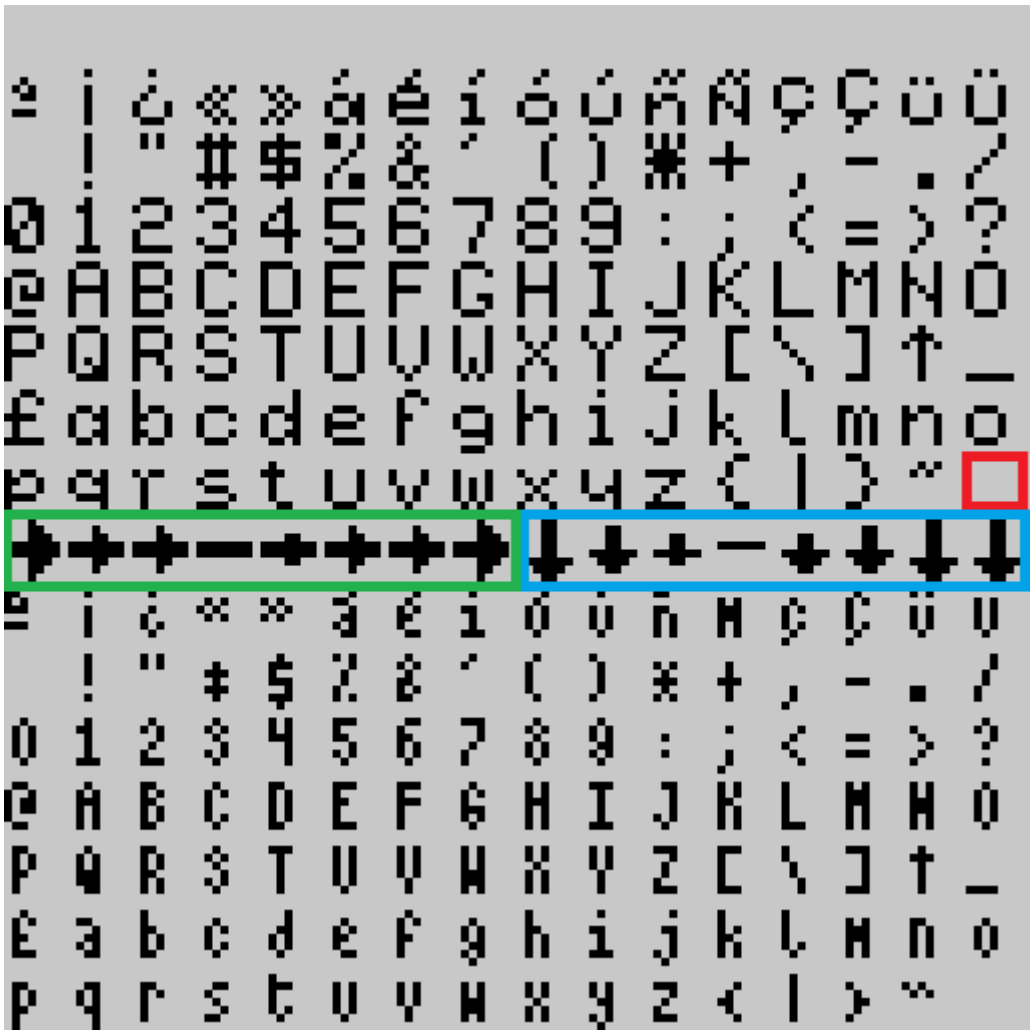

Figure 3: Special characters

image or track was not loaded during compilation. It also happens when a RETURN is executed without a prior GOSUB.
- System Error 2: Too many options have been created, the limit of possible options has been exceeded.
- System Error 3: There are no options available, a `CHOOSE` command has been launched without having any `OPTION` before.
- System Error 4: The file with the music module to load is too large, it must be less than 16Kib.
- System Error 5: There is no music module loaded to play.
- System Error 6: Invalid instruction code.
- System Error 7: Access to array position out of range.
- System Error 8: Missing option. When scrolling, the declared options shift upwards. If one of them goes over the top of the margins, this error is generated.

Disk errors are errors that could be caused when the game engine accesses the disk, and correspond to +3DOS errors:

- Disk Error 0: Drive not ready
- Disk Error 1: Disk is write protected
- Disk Error 2: Seek fail
- Disk Error 3: CRC data error
- Disk Error 4: No data
- Disk Error 5: Missing address mark
- Disk Error 6: Unrecognised disk format
- Disk Error 7: Unknown disk error
- Disk Error 8: Disk changed whilst +3DOS was using it
- Disk Error 9: Unsuitable media for drive
- Disk Error 20: Bad filename
- Disk Error 21: Bad parameter
- Disk Error 22: Drive not found
- Disk Error 23: File not found
- Disk Error 24: File already exists
- Disk Error 25: End of file
- Disk Error 26: Disk full
- Disk Error 27: Directory full
- Disk Error 28: Read-only file
- Disk Error 29: File number not open (or open with wrong access)
- Disk Error 30: Access denied (file is in use already)
- Disk Error 31: Cannot rename between drives
- Disk Error 32: Extent missing (which should be there)
- Disk Error 33: Uncached (software error)
- Disk Error 34: File too big (trying to read or write past 8 megabytes)
- Disk Error 35: Disk not bootable (boot sector is not acceptable to DOS BOOT)
- Disk Error 36: Drive in use (trying to re-map or remove a drive with files open)

These errors appear when accessing the disk, when searching for more pieces of text, images, etc. If error 23 appears (File not found), it is usually because you have forgotten to include a necessary

file on the disk. Other errors already indicate an error with the disk drive or the disk itself.

1.24 Acknowledgements

- David Beazley by PLY
- Einar Saukas for the compressor ZX0 and ZX7.
- Mokona for their version of the ZX0 compressor for Python.
- Sylvain Glaize for the decompressor version ZX0 for Python.
- DjMorgul for the abbreviation finder, adapted from Daad Reborn Tokenizer
- Shiru by BeepFx.
- Seasip for mkp3fs from Taptools.
- Sergey.V.Bulba for the Vortex Tracker player.
- Augusto Ruiz for the WyzTracker player.
- To the team behind the sjasmplus assembler.
- Tranqui69 for the logo and support.
- XimoKom, Javier Fopiani, and Fran Kapilla for their invaluable help testing the engine.
- Pablo Martínez Merino for help with Linux testing and examples.
- Sergio ThePoPe for getting me interested in Plus3.
- El_Mesías, Arnau Jess, and Javi Ortiz for sharing the story.
- To the AD Adventure Club CAAD for their support.
- To all the members of the `Choose your Destiny` Telegram group.

1.25 Licenses

This software package is subject to different licenses depending on its components:

- The compiler is licensed under the GNU Affero General Public License v3.0.
- The interpreter (the executable portion on the target machine) is licensed under the following license:

```
Choose Your Destiny

Copyright (c) 2025 Sergio Chico (Cronomantic)

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and assoc

- The above copyright notice and this permission notice shall be included in all copies or substantial

- The above copyright notice and/or one of the project logos must be prominently displayed both on the

- THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT
```

- PyZX0 is licensed under a BSD-3.
- The WyzTracker player is licensed under the MIT license.
- PLY and the WyzTracker player do not have specific licenses; a license similar to BSD or MIT

is assumed.

- BeepFx is licensed under the WTFPL v.2.

In simple terms, this means that if you use this engine to make a game, you must always indicate on the loading screen or within the game itself that it was made with `CYD` or use one of the project logos (included in the `assets` directory). You must also do so on the download website if you distribute the game online, and on the box or media container if you distribute it physically. Aside from the above condition, the game you create **ALWAYS** remains your property and authorship; you can sell or distribute it without having to publish the source code or artwork. However, the tool is licensed under the AGPL, which means that if you modify it or create your own version, you are required to publish the code and indicate that it is based on this project.